



BIG DATA ANALYTICS ON E-COMMERCE DATASET USING APACHE SPARK: PERFORMANCE AND OPTIMIZATION STUDY

A Comparative Study of RDD, DataFrame, and Spark SQL with
Optimization Techniques

Academic Year: 2025–2026



Team members :

Mohamed Ehab 202201236

Yousef Selim | 202201255

Table of Contents

Big Data Analytics on E-Commerce Dataset Using Apache Spark: Performance and Optimization Study	1
Academic Year: 2025–2026	1
1. Problem Statement	3
2.1 Dataset Source	4
2.2 Dataset Overview	4
2.4.1 Order and Transaction Data	5
2.6 Dataset Justification	7
3.1 Data Cleaning	8
Handling Missing Values	8
Duplicate Removal	8
Data Type Conversion	8
3.3 Exploratory Data Analysis (EDA)	8
3.4 Importance of Preprocessing	9
RDD API (Low-Level)	10
6. Execution Plan Analysis	13
6.1 Overview	14
6.2 Performance Results	15
6.3 Key Observations	16
1. RDD Performance	16
3. Spark SQL Performance	17
7. Optimization Analysis	17
7.1 Overview	17
Broadcast Join	18
7.3 Caching	18
CSV Format	19
Parquet Format	20
8. Cluster Configuration and Scalability Tests.	20
9. Conclusion	21
9.1 Best Performing API	21
Cluster Configuration and Scalability Tests	83

1. Problem Statement

Modern e-commerce platforms generate massive volumes of transactional, customer, product, and behavioral data. Each interaction, such as purchases, payments, returns, and reviews, produces valuable information that can be analyzed to understand business performance and customer behavior. However, as data size grows beyond the capacity of traditional single-machine systems, efficient analysis becomes a Big Data challenge that requires distributed processing.

This project focuses on analyzing a large-scale global e-commerce dataset containing more than one million records. The dataset includes order details, customer demographics, product categories, payment methods, geographic information, and financial metrics. Due to its size and diversity, it is well-suited for evaluating Apache Spark's distributed data processing capabilities.

The central problem addressed in this project is:

How can Apache Spark efficiently analyze large-scale e-commerce transaction data, and how do RDD, DataFrame, and Spark SQL APIs differ in terms of performance, execution plans, and optimization?

This problem goes beyond basic descriptive analysis. The project explores meaningful analytical tasks such as identifying high-value customers, analyzing revenue across categories and countries, understanding purchasing patterns over time, and evaluating customer behavior across different segments.

A key objective of the project is to compare three Spark programming approaches:

-RDD API: A low-level abstraction that provides full control over distributed data processing. However, it lacks automatic optimization and often requires manual transformations, which can lead to slower execution.

-DataFrame API: A structured, schema-based abstraction that allows Spark to apply internal optimizations such as predicate pushdown, column pruning, and efficient aggregation.

-Spark SQL: A declarative interface that enables querying data using SQL syntax while leveraging the same optimization engine as DataFrames, often resulting in better readability and performance.

Therefore, this project addresses both a **business analytics problem** and a **technical performance evaluation problem**.

By implementing the same analytical queries using RDDs, DataFrames, and Spark SQL, the project provides a comprehensive comparison of Spark APIs and demonstrates why structured approaches are generally more efficient for large-scale data analytics.

2. Dataset Description

2.1 Dataset Source

The dataset used in this project is publicly available on Kaggle:

Global E-Commerce Dataset (+1M Records, 2024–2026)

<https://www.kaggle.com/datasets/akrambelha/global-e-commerce-dataset-1m-records-20242026>

This dataset was selected because it provides a large-scale, realistic simulation of e-commerce operations, making it suitable for Big Data analytics and performance evaluation using Apache Spark.

2.2 Dataset Overview

The dataset is a large-scale synthetic dataset that simulates real-world e-commerce transactions. It includes detailed information about orders, customers, products, payments, logistics, and user behavior.

-Number of Records: 1,000,123 rows

-Number of Columns: 62 attributes

-File Formats: CSV and Excel

-Dataset Type: Structured, tabular data

The dataset satisfies the project requirement of handling **datasets with more than one million records**, making it appropriate for evaluating distributed data processing techniques.

2.3 Temporal Coverage

The dataset spans a time period of approximately two years:

- **Start Date:** February 2024
- **End Date:** February 2026

This temporal range enables the implementation of **time-based analytical queries**, such as monthly trends and seasonal patterns.

2.4 Data Schema and Feature Categories

The dataset contains a wide range of attributes that can be grouped into the following categories:

2.4.1 Order and Transaction Data

- order_id, order_date, order_status
- order_year, order_month, order_day
- order_hour, order_minute, order_second
- is_weekend, order_priority

These attributes describe transaction-level details and allow temporal and operational analysis.

2.4.2 Customer Information

- customer_id, customer_name
- gender, age
- customer_segment
- country, city
- customer_loyalty_score, total_orders_by_customer
- account_creation_date

These features enable customer segmentation and behavioral analysis.

2.4.3 Product Information

- product_id, product_name
- category, sub_category
- brand
- product_rating_avg, product_reviews_count
- stock_quantity

These attributes support product-level analytics such as category performance and brand analysis.

2.4.4 Financial and Pricing Data

- unit_price_usd, quantity
- discount_percent, discount_amount_usd
- total_price_usd

- cost_usd, profit_usd, profit_margin_percent
- tax_usd, currency

These fields are essential for revenue analysis, profit calculation, and financial evaluation.

2.4.5 Payment and Shipping Information

- payment_method, payment_status
- installment_plan
- shipping_method, shipping_cost_usd
- delivery_days, delivery_status
- shipping_country, warehouse_location

These attributes allow analysis of logistics performance and payment behavior.

2.4.6 Marketing and User Behavior

- campaign_source, traffic_source
- device_type
- session_duration_minutes
- pages_visited, abandoned_cart_before

These features provide insights into customer interaction and marketing effectiveness.

2.4.7 Customer Feedback and Risk Indicators

- rating, review_sentiment
- customer_feedback
- fraud_risk_score
- return_reason

These attributes support sentiment analysis, fraud detection, and return behavior analysis

2.5 Data Characteristics

The dataset has several characteristics that make it suitable for Big Data analytics:

- **Large Volume:** Over one million records
- **High Dimensionality:** 62 attributes
- **Mixed Data Types:** Numerical, categorical, and temporal data
- **Realistic Structure:** Simulates real-world e-commerce workflows
- **Synthetic Nature:** Generated data with no privacy concerns

2.6 Dataset Justification

The selected dataset is highly suitable for distributed processing using Apache Spark due to its scale, structure, and analytical potential.

First, the dataset contains more than one million records, which exceeds the processing capacity of traditional single-machine approaches for efficient analysis. This makes distributed computing necessary to ensure scalability, faster execution, and efficient resource utilization.

Second, the dataset includes a wide range of attributes covering multiple aspects of an e-commerce system, such as transactions, customers, products, payments, logistics, and user behavior. This diversity enables the implementation of complex analytical queries, including aggregations, joins, window functions, and subqueries, which are essential for evaluating Spark's performance and optimization capabilities.

In addition, the dataset supports meaningful business insights, such as:

- Revenue analysis across product categories and countries
- Identification of high-value customers and purchasing patterns
- Analysis of payment methods and order distributions
- Time-based trends in sales and customer activity

These analytical capabilities align directly with the objectives of the project, which aim to combine **business intelligence** with **performance evaluation of distributed processing systems**.

2.7 Big Data Characteristics

The dataset exhibits key characteristics of Big Data:

- **Volume:** Contains over one million records, requiring distributed storage and processing
- **Variety:** Includes multiple data types such as categorical, numerical, and temporal data
- **Velocity (Simulated):** Represents continuous generation of transactional data over time
- **Complexity:** Involves multiple interconnected entities, including customers, products, and orders

These characteristics make the dataset highly suitable for evaluating distributed data processing using Apache Spark and demonstrate its applicability to real-world Big Data analytics scenarios.

3. Data Preprocessing

3.1 Data Cleaning

Before performing analytical queries, the dataset was preprocessed to ensure data quality and consistency. This step is essential when working with large-scale datasets, as raw data may contain missing values, inconsistencies, or duplicate records.

The following preprocessing steps were applied:

Handling Missing Values

Records with missing values in critical numerical columns, particularly `total_price_usd`, were removed. This ensures that aggregation operations such as sum and average produce accurate and reliable results.

Duplicate Removal

Duplicate records were eliminated using Spark's `dropDuplicates()` function. This guarantees that each transaction is counted only once and prevents bias in analytical outputs.

Data Type Conversion

The `order_date` column was converted into a proper timestamp format. This enables time-based analysis such as monthly aggregation and window functions.

3.2 Feature Preparation

To support analytical queries, additional transformations were performed:

- A new month column was derived from `order_date` using date formatting functions. This allows grouping orders by month for trend analysis.

- Key analytical features such as `total_price_usd`, `category`, and `country` were validated and prepared for filtering and aggregation operations.

3.3 Exploratory Data Analysis (EDA)

Initial exploration of the dataset was conducted to understand its structure and distribution:

-The dataset contains more than 1 million records, confirming its suitability for distributed processing.

-The schema includes a mix of:

- Numerical data (e.g., prices, profits)
- Categorical data (e.g., category, country)
- Temporal data (e.g., timestamps).

3.4 Importance of Preprocessing

Preprocessing improves both data quality and system performance:

- Removing null values ensures accurate calculations
- Eliminating duplicates prevents data redundancy
- Converting data types enables efficient query execution
- Preparing features simplifies downstream analysis

After preprocessing, the dataset became clean, consistent, and ready for efficient distributed processing using Apache Spark. This ensured that all subsequent analytical queries produced reliable and meaningful results.

4. Queries Implemented

The project includes **ten analytical queries** implemented using RDD, DataFrame, and Spark SQL APIs. These queries cover filtering, aggregation, grouping, sorting, window functions, and subqueries.

Query 1: Filter Completed Orders Above 500

Filters completed orders where the total price exceeds 500 USD.

Query 2: Total Revenue by Category

Calculates total revenue for each product category.

Query 3: Average Order Value by Country

Computes the average order value for each country.

Query 4: Orders by Payment Method

Counts the number of orders per payment method.

Query 5: Maximum Order per Customer

Identifies the highest-value order for each customer.

Query 6: Orders per Month

Analyzes order distribution over time.

Query 7: Revenue by Category and Country

Calculates revenue grouped by both category and country.

Query 8: Distinct Products per Category

Counts distinct products within each category.

Query 9: Cumulative Revenue (Window Function)

Calculates cumulative revenue over time using a window function.

Query 10: High-Value Customers (Subquery)

Identifies customers whose total spending exceeds the average.

5. Implementation Overview

In this project, each analytical query was implemented using three different Apache Spark APIs to evaluate their performance, flexibility, and optimization capabilities.

5.1 Spark APIs Used

RDD API (Low-Level)

The Resilient Distributed Dataset (RDD) API provides a low-level programming interface for distributed data processing. It allows full control over transformations and actions, enabling custom data manipulation using functional programming concepts such as `map`, `filter`, and `reduce`.

However, RDD does not leverage Spark's Catalyst Optimizer, meaning that operations are executed as defined without automatic optimization. As a result, RDD implementations tend to involve more manual coding and generally exhibit lower performance compared to structured APIs.

DataFrame API (Structured)

The DataFrame API provides a higher-level abstraction built on top of RDDs, with a defined schema and column-based operations. This structured representation allows Spark to internally optimize queries using the Catalyst Optimizer and Tungsten execution engine.

DataFrames support efficient operations such as filtering, grouping, aggregation, and joins, while benefiting from:

- **Predicate pushdown** (filtering data early)
- **Column pruning** (reading only necessary columns)
- **Optimized aggregation and join strategies**

This results in significantly improved performance and reduced resource usage compared to RDD.

Spark SQL (Declarative)

Spark SQL provides a declarative interface for querying data using SQL syntax. It operates on structured data and uses the same underlying optimization engine as the DataFrame API.

By allowing queries to be expressed in SQL, Spark SQL improves readability and maintainability while still achieving high performance. It also enables easier analysis of execution plans and is often the most efficient approach for complex analytical queries.

5.2 Analysis Performed

To evaluate the effectiveness of each API, several aspects were analyzed for every query:

Execution Plan Analysis

Execution plans were generated using:

```
.explain(True)
```

This provided detailed insights into the different stages of query execution:

- **Logical Plan:** Represents the initial query structure

- **Optimized Logical Plan:** Shows transformations applied by the Catalyst Optimizer
- **Physical Plan:** Describes how the query is executed across the cluster

Analyzing these plans helped in understanding how Spark optimizes operations such as filtering, aggregation, and joins.

Performance Evaluation

Execution time was measured for each query across all three APIs (RDD, DataFrame, and SQL). This allowed a direct comparison of performance and highlighted the impact of Spark's optimization mechanisms.

The results were used to identify:

- The most efficient API for each type of operation
- Performance differences between low-level and structured APIs
- The effect of query complexity on execution time

Optimization Techniques

Several optimization strategies were applied and evaluated to improve performance:

-Caching: Frequently used datasets were cached in memory to avoid recomputation and reduce execution time for repeated queries.

-Partitioning: Data was repartitioned to improve parallelism and reduce data skew, which helps optimize distributed execution.

-Broadcast Joins: Smaller datasets were broadcast to all worker nodes to avoid costly shuffle operations during joins.

-File Format Optimization (CSV vs Parquet): The dataset was converted from CSV to Parquet format. Parquet's columnar storage and compression significantly reduced I/O operations and improved query performance.

By implementing each query using multiple Spark APIs and applying various optimization techniques, this project provides a comprehensive evaluation of Spark's processing capabilities. The combination of execution plan analysis and performance measurement offers clear insights into how different approaches impact efficiency in large-scale data processing.

6. Execution Plan Analysis

Apache Spark provides detailed execution plans for structured APIs such as DataFrame and Spark SQL. These plans are generated using the `.explain(True)` function and are essential for understanding how queries are optimized and executed in a distributed environment.

Each query execution is divided into three main stages: **Logical Plan**, **Optimized Logical Plan**, and **Physical Plan**.

6.1 Logical Plan

The Logical Plan represents the initial structure of the query as written by the user. It describes the sequence of operations, such as filtering, projection, grouping, and aggregation, without considering performance optimization.

At this stage:

- Spark interprets the query syntax
- Identifies required columns and operations
- Builds a high-level representation of the computation

The Logical Plan reflects *what* operations need to be performed, but not *how* they will be executed.

6.2 Optimized Logical Plan

The Optimized Logical Plan is generated by Spark's **Catalyst Optimizer**, which applies a series of rule-based transformations to improve query efficiency.

Key optimization techniques observed in this project include:

- **Predicate Pushdown:**
Filters are applied as early as possible during data reading to reduce the amount of data processed.
- **Column Pruning:**
Only the required columns are selected, minimizing memory usage and I/O operations.
- **Filter Optimization:**
Redundant or unnecessary conditions are simplified or removed.
- **Aggregation Optimization:**
Partial aggregations are performed before shuffle operations to reduce data movement.

These optimizations significantly reduce execution time and improve resource utilization.

6.3 Physical Plan

The Physical Plan defines how the query is actually executed across the distributed environment. It translates the optimized logical operations into concrete execution strategies.

Common execution operators observed include:

- **HashAggregate:**
Used for grouping and aggregation operations. It performs partial and final aggregation efficiently.
- **SortMergeJoin:**
Used for joining large datasets. It involves sorting both datasets before merging.
- **Exchange (Shuffle):**
Represents data movement across partitions. It is one of the most expensive operations and is minimized through optimization.
- **Scan Operations:**
Reading data from storage (CSV or Parquet).

The Physical Plan reflects *how* Spark executes the query and is critical for understanding performance behavior.

6.4 RDD Execution Behavior

Unlike DataFrame and Spark SQL, the RDD API does not generate logical or physical execution plans. Instead, it provides a **lineage graph** of transformations.

This means:

- No automatic optimization is applied
- Operations are executed as defined by the user
- Performance depends heavily on manual implementation

As a result, RDD-based queries are generally less efficient compared to structured APIs.

6. Performance Comparison

6.1 Overview

To evaluate the efficiency of different Apache Spark APIs, the execution time of each query was measured across three implementations:

- RDD API
- DataFrame API
- Spark SQL

This comparison provides insights into how Spark's optimization mechanisms affect performance when processing large-scale datasets.

6.2 Performance Results

The following table summarizes the execution time (in seconds) for all ten queries:

Query	RDD (s)	DataFrame (s)	SQL (s)
Q1	104.0	10.7	8.8
Q2	112.6	12.1	9.5
Q3	108.8	10.0	8.3
Q4	105.5	9.9	8.1
Q5	127.1	13.2	11.0
Q6	109.3	9.8	9.2

Q7	144.1	9.8	8.0
Q8	136.5	13.9	11.1
Q9	233.6	17.0	13.2
Q10	160.3	33.8	11.5

6.3 Key Observations

1. RDD Performance

RDD consistently shows the highest execution time across all queries.

- Typically 10x to 20x slower than DataFrame and SQL
- Requires manual transformations
- Does not benefit from Spark's optimization engine

This confirms that RDD is less suitable for large-scale analytical workloads.

2. DataFrame Performance

DataFrame performs significantly better than RDD.

- Uses structured data representation
- Benefits from Catalyst Optimizer
- Applies optimizations such as:
 - Predicate pushdown
 - Column pruning
 - Efficient aggregation

DataFrame offers a good balance between performance and flexibility.

3. Spark SQL Performance

Spark SQL consistently provides the best performance across all queries.

- Declarative syntax simplifies query design
- Uses the same optimization engine as DataFrames
- Often produces the most efficient execution plans

SQL is particularly effective for complex analytical queries.

6.4 Impact of Query Complexity

Execution time varies depending on the type of query:

- Simple queries (filtering, counting): Lower execution time (Q1–Q4)
- Aggregation queries: Moderate execution time due to grouping and shuffle (Q2, Q3, Q7)
- Advanced queries (window functions, subqueries): Higher execution time due to sorting and multiple stages (Q9, Q10)

Query complexity directly affects performance, especially in distributed systems.

6.5 Performance Gap Analysis

The performance difference between APIs can be explained by:

- Optimization: DataFrame and SQL use Catalyst Optimizer, while RDD does not
 - Execution Planning: Structured APIs generate optimized execution plans
 - Data Handling: Column-based processing reduces unnecessary data access
 - Shuffle Reduction: Optimized aggregation reduces data movement across partitions
-

7. Optimization Analysis

7.1 Overview

In addition to implementing analytical queries, several optimization techniques were applied to evaluate their impact on performance. These optimizations are essential in distributed systems like Apache Spark, where inefficient execution can lead to increased computation time and resource usage.

The following optimization strategies were analyzed:

- Join optimization (Broadcast vs Sort-Merge Join)
- Caching
- Partitioning
- File format optimization (CSV vs Parquet)

7.2 Join Optimization

Broadcast Join

Broadcast join is used when one of the datasets is small. In this approach, the smaller dataset is broadcast to all worker nodes, avoiding shuffle operations.

Advantages:

- Eliminates expensive data shuffling
- Faster join execution

Observation:

Broadcast join showed improved performance when joining small subsets of data.

Sort-Merge Join

Sort-Merge Join is used when both datasets are large. It involves sorting both datasets on the join key and then merging them.

Characteristics:

- Requires shuffle operations
- More expensive than broadcast join
- Suitable for large-scale joins

Observation:

When the broadcast was disabled, Spark used Sort-Merge Join, which resulted in slightly higher execution time.

7.3 Caching

Caching stores intermediate data in memory, allowing Spark to reuse it without recomputation.

Results:

Scenario	Execution Time
Without Cache	~16.3 s
With Cache	~3.8 s

Impact:

- Significant reduction in execution time
- Avoids repeated computation of the same dataset

Caching is highly effective for queries that reuse the same data multiple times.

7.4 Partitioning

Partitioning controls how data is distributed across nodes.

Two approaches were evaluated:

- **Default partitioning:** Uses Spark's automatic partitioning
- **Repartitioning:** Manually redistributes data

Observations:

- Proper partitioning improves parallelism
- Excessive repartitioning can introduce overhead
- Data skew can negatively affect performance

7.5 File Format Optimization (CSV vs Parquet)

The dataset was originally stored in CSV format and later converted to Parquet format.

CSV Format

- Row-based storage

- No compression
- Higher I/O cost

Parquet Format

- Columnar storage
- Efficient compression
- Faster data access

Results:

Format	Execution Time
CSV	~24 s
Parquet	~1.3 s

Parquet significantly improves performance due to reduced data reading and optimized storage.

8. Cluster Configuration and Scalability Tests

To evaluate system scalability, different Spark configurations were tested by varying the number of partitions and memory allocation.

The results show that increasing partitions improves parallelism but may introduce overhead if too large. Similarly, increasing memory improves performance up to a certain limit, after which the improvement becomes minimal.

These experiments demonstrate how Spark performance depends on cluster configuration and resource allocation.

9. Conclusion

This project demonstrated the effectiveness of Apache Spark in processing and analyzing large-scale e-commerce data. By implementing ten analytical queries using RDD, DataFrame, and Spark SQL APIs, the project provided a comprehensive comparison of their performance and execution behavior.

The results showed that DataFrame and Spark SQL significantly outperform RDD, primarily due to Spark's Catalyst Optimizer and efficient execution strategies. While RDD offers flexibility and low-level control, it lacks optimization capabilities, making it less suitable for large-scale analytical workloads.

In addition, the application of optimization techniques such as caching, broadcast joins, partitioning, and Parquet storage resulted in substantial performance improvements. Among these, caching and the Parquet format had the most noticeable impact in reducing execution time.

From a business perspective, the analysis provided valuable insights into customer behavior, revenue distribution, and sales trends. From a technical perspective, it highlighted the importance of choosing the appropriate API and applying optimization strategies in Big Data systems.

In conclusion, this project confirms that structured APIs (DataFrame and Spark SQL) combined with proper optimization techniques provide the most efficient and scalable solution for large-scale data processing using Apache Spark.

9. Final Insights

This project provides both analytical and technical insights derived from processing a large-scale e-commerce dataset using Apache Spark.

9.1 Best Performing API

Among the three APIs, Spark SQL consistently achieved the best performance, followed closely by the DataFrame API, while RDD showed the slowest execution.

This is because:

- Spark SQL and DataFrame use the Catalyst Optimizer, which automatically improves query execution
- They generate optimized logical and physical plans

-They reduce unnecessary data processing through techniques such as predicate pushdown and column pruning

In contrast, RDD lacks automatic optimization and executes operations as defined, leading to higher execution time.

9.2 Business Insights

The analysis of the dataset reveals several important patterns:

-A small group of customers contributes significantly to overall revenue, indicating high-value customers

-Revenue distribution varies across categories and countries, showing differences in demand and market behavior

-Customer purchasing behavior changes over time, suggesting temporal trends and possible seasonality

-Payment methods and order patterns vary across regions, reflecting customer preferences

9.3 Lessons Learned (Spark Optimization)

This project highlights several key lessons in Spark optimization:

- Structured APIs (DataFrame and SQL) are more efficient and scalable than RDD
 - Caching significantly reduces execution time for repeated queries
 - Broadcast joins improve performance when working with small datasets
 - Parquet format greatly enhances performance compared to CSV due to columnar storage
 - Proper partitioning improves parallelism but must be carefully managed to avoid overhead
-

Screenshots Evidences

SHUFFLE SIZE AND MEMORY USAGE RDD

7

reduceByKey at /home/yousef/spark_project/rdd/queries_rdd.py:26

(kill) 2026/04/24 18:06:51
+details

9 s

0/4 (1 running)

Page: 1

1 Pages. Jump to 1 . Show 100 items in a page. Go

▼ Pending Stages (1)

Page: 1

1 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id ▼	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
8	sortBy at /home/yousef/spark_project/rdd/queries_rdd.py:27	+details Unknown	Unknown	0/4				

Page: 1

1 Pages. Jump to 1 . Show 100 items in a page. Go

▼ Completed Stages (6)

Page: 1

1 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id ▼	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
6	count at /home/yousef/spark_project/rdd/queries_rdd.py:17	2026/04/24 18:04:01	2.8 min	4/4	391.4 MiB			
5	showString at NativeMethodAccessorImpl.java:0	2026/04/24 18:03:57	1 s	1/1	64.0 KiB			
4	count at NativeMethodAccessorImpl.java:0	2026/04/24 18:03:52	1.0 s	1/1			236.0 B	
2	count at NativeMethodAccessorImpl.java:0	2026/04/24 18:03:45	6 s	4/4	391.6 MiB			236.0 B
1	csv at NativeMethodAccessorImpl.java:0	2026/04/24	1.5 min	4/4	391.6			

RDD RESULTS

Spark Jobs (?)

User: yousef
Total Uptime: 30 s
Scheduling Mode: FIFO
Active Jobs: 1
Completed Jobs: 1

▶ Event Timeline

▼ Active Jobs (1)

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

Job Id ▼	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
1	csv at NativeMethodAccessorImpl.java:0 csv at NativeMethodAccessorImpl.java:0 (kill)	2026/04/24 01:44:03	6 s	0/1	0/4 (1 running)

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

▼ Completed Jobs (1)

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

Job Id ▼	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
0	csv at NativeMethodAccessorImpl.java:0 csv at NativeMethodAccessorImpl.java:0	2026/04/24 01:44:01	2 s	1/1	1/1

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

Spark Jobs (?)

User: yousef
Total Uptime: 9.1 min
Scheduling Mode: FIFO
Active Jobs: 1
Completed Jobs: 14

▶ Event Timeline

▼ Active Jobs (1)

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

Job Id ▼	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
14	sortBy at /home/yousef/spark_project/rdd/queries_rdd.py:44 sortBy at /home/yousef/spark_project/rdd/queries_rdd.py:44 (kill)	2026/04/24 01:52:04	42 s	0/2	1/8 (1 running)

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

Terminal

▼ Completed Jobs (14)

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

Job Id ▼	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
13	runJob at PythonRDD.scala:181 runJob at PythonRDD.scala:181	2026/04/24 01:52:03	1 s	1/1 (2 skipped)	3/3 (8 skipped)
12	runJob at PythonRDD.scala:181 runJob at PythonRDD.scala:181	2026/04/24 01:52:01	2 s	2/2 (1 skipped)	5/5 (4 skipped)
11	sortBy at /home/yousef/spark_project/rdd/queries_rdd.py:35 sortBy at /home/yousef/spark_project/rdd/queries_rdd.py:35	2026/04/24 01:52:00	1.0 s	1/1 (1 skipped)	4/4 (4 skipped)

Thunderbird Mail Spark Jobs (?)

User: yousef
Total Uptime: 14 min
Scheduling Mode: FIFO
Active Jobs: 1
Completed Jobs: 20

Event Timeline

Active Jobs (1)

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
20	sortBy at /home/yousef/spark_project/rd/queries_rdd.py:62 sortBy at /home/yousef/spark_project/rd/queries_rdd.py:62 (kill)	2026/04/24 01:56:02	1.6 min	0/2	2/8 (1 running)

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

Completed Jobs (20)

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
19	runJob at PythonRDD.scala:181 runJob at PythonRDD.scala:181	2026/04/24 01:55:53	8 s	2/2 (1 skipped)	5/5 (4 skipped)
18	sortBy at /home/yousef/spark_project/rd/queries_rdd.py:53 sortBy at /home/yousef/spark_project/rd/queries_rdd.py:53	2026/04/24 01:55:50	3 s	1/1 (1 skipped)	4/4 (4 skipped)
17	sortBy at /home/yousef/spark_project/rd/queries_rdd.py:53 sortBy at /home/yousef/spark_project/rd/queries_rdd.py:53	2026/04/24 01:54:11	1.7 min	2/2	8/8

Spark Jobs (?)

User: yousef
Files Uptime: 16 min
Scheduling Mode: FIFO
Active Jobs: 1
Completed Jobs: 26

Event Timeline

Active Jobs (1)

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
26	sortBy at /home/yousef/spark_project/rd/queries_rdd.py:82 sortBy at /home/yousef/spark_project/rd/queries_rdd.py:82 (kill)	2026/04/24 01:59:22	19 s	0/3	0/12 (1 running)

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

Completed Jobs (26)

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
25	runJob at PythonRDD.scala:181 runJob at PythonRDD.scala:181	2026/04/24 01:59:22	0.8 s	2/2 (1 skipped)	5/5 (4 skipped)
24	sortBy at /home/yousef/spark_project/rd/queries_rdd.py:71 sortBy at /home/yousef/spark_project/rd/queries_rdd.py:71	2026/04/24 01:59:21	0.6 s	1/1 (1 skipped)	4/4 (4 skipped)
23	sortBy at /home/yousef/spark_project/rd/queries_rdd.py:71 sortBy at /home/yousef/spark_project/rd/queries_rdd.py:71	2026/04/24 01:57:48	1.5 min	2/2	8/8

QUERY 1

```
13
14 # --- Q1: Filter orders above $500 with status = Completed ---
15 t = time.time()
16 q1 = rdd.filter(lambda r: r['order_status'] == 'Completed' and r['total_price_usd'] is not None
    and float(r['total_price_usd']) > 500)
17 results['Q1_count'] = q1.count()
18 results['Q1_time'] = time.time() - t
19 print("\n=== RDD LINEAGE (Execution Flow) ===")
20 print(q1.toDebugString())
21 print(f"Q1 RDD | Count={results['Q1_count']} | Time={results['Q1_time']:.3f}s")
```

```
=== RDD LINEAGE (Execution Flow) ===
```

```
b'(4) PythonRDD[27] at RDD at PythonRDD.scala:53 []\n | MapPartitionsRDD[25] at
  javaToPython at NativeMethodAccessorImpl.java:0 []\n | MapPartitionsRDD[24] at
  javaToPython at NativeMethodAccessorImpl.java:0 []\n | SQLExecutionRDD[23] at
  javaToPython at NativeMethodAccessorImpl.java:0 []\n | MapPartitionsRDD[22] at
  javaToPython at NativeMethodAccessorImpl.java:0 []\n | FileScanRDD[21] at javaT
  oPython at NativeMethodAccessorImpl.java:0 []'
```

```
Q1 RDD | Count=198151 | Time=148.184s
```

QUERY 2

```
3 # --- Q2: Total revenue by category ---
4 t = time.time()
5 q2 = rdd.map(lambda r: (r['category'], float(r['total_price_usd']) if r['total_price_usd'] else
6 0)) \
7     .reduceByKey(lambda a, b: a + b) \
8     .sortBy(lambda x: -x[1])
9 results['Q2'] = q2.take(10)
10 results['Q2_time'] = time.time() - t
11 print("\n=== RDD LINEAGE (Execution Flow) ===")
12 print(q2.toDebugString())
13 print(f"Q2 RDD | Top category: {results['Q2'][0]} | Time={results['Q2_time']:.3f}s")
```

```
=== RDD LINEAGE (Execution Flow) ===
b'(4) PythonRDD[40] at RDD at PythonRDD.scala:53 []\n | MapPartitionsRDD[37] at
mapPartitions at PythonRDD.scala:160 []\n | ShuffledRDD[36] at partitionBy at
NativeMethodAccessorImpl.java:0 []\n +- (4) PairwiseRDD[35] at sortBy at /home/you
sef/spark_project/rdd/queries_rdd.py:27 []\n | PythonRDD[34] at sortBy at /
home/yousef/spark_project/rdd/queries_rdd.py:27 []\n | MapPartitionsRDD[31]
at mapPartitions at PythonRDD.scala:160 []\n | ShuffledRDD[30] at partitionB
y at NativeMethodAccessorImpl.java:0 []\n +- (4) PairwiseRDD[29] at reduceByKe
y at /home/yousef/spark_project/rdd/queries_rdd.py:26 []\n | PythonRDD[28
] at reduceByKey at /home/yousef/spark_project/rdd/queries_rdd.py:26 []\n
| MapPartitionsRDD[25] at javaToPython at NativeMethodAccessorImpl.java:0 []\n
| MapPartitionsRDD[24] at javaToPython at NativeMethodAccessorImpl.java:0
[]\n | SQLExecutionRDD[23] at javaToPython at NativeMethodAccessorImpl.j
ava:0 []\n | MapPartitionsRDD[22] at javaToPython at NativeMethodAccessor
Impl.java:0 []\n | FileScanRDD[21] at javaToPython at NativeMethodAccesso
rImpl.java:0 []'
Q2 RDD | Top category: ('Electronics', 139865986.940000006) | Time=169.519s
```


QUERY 3

```
33
34 # --- Q3: Average order value by country ---
35 t = time.time()
36 q3 = rdd.map(lambda r: (r['country'], (float(r['total_price_usd']) if r['total_price_usd'] else
    0, 1))) \
37     .reduceByKey(lambda a, b: (a[0]+b[0], a[1]+b[1])) \
38     .mapValues(lambda x: round(x[0]/x[1], 2)) \
39     .sortBy(lambda x: -x[1])
40 results['Q3'] = q3.take(10)
41 results['Q3_time'] = time.time() - t
42 print("\n=== RDD LINEAGE (Execution Flow) ===")
43 print(q3.toDebugString())
44 print(f"Q3 RDD | Top country avg: {results['Q3'][0]} | Time={results['Q3_time']:.3f}s")
```

```
=== RDD LINEAGE (Execution Flow) ===
b'(4) PythonRDD[53] at RDD at PythonRDD.scala:53 []\n | MapPartitionsRDD[50] at
  mapPartitions at PythonRDD.scala:160 []\n | ShuffledRDD[49] at partitionBy at
NativeMethodAccessorImpl.java:0 []\n +- (4) PairwiseRDD[48] at sortBy at /home/you
usef/spark_project/rdd/queries_rdd.py:39 []\n   | PythonRDD[47] at sortBy at /
home/yousef/spark_project/rdd/queries_rdd.py:39 []\n   | MapPartitionsRDD[44]
at mapPartitions at PythonRDD.scala:160 []\n   | ShuffledRDD[43] at partitionB
y at NativeMethodAccessorImpl.java:0 []\n   +- (4) PairwiseRDD[42] at reduceByKe
ware /home/yousef/spark_project/rdd/queries_rdd.py:37 []\n      | PythonRDD[41]
at reduceByKey at /home/yousef/spark_project/rdd/queries_rdd.py:37 []\n      |
  MapPartitionsRDD[25] at javaToPython at NativeMethodAccessorImpl.java:0 []\n
      | MapPartitionsRDD[24] at javaToPython at NativeMethodAccessorImpl.java:0
[]\n      | SQLExecutionRDD[23] at javaToPython at NativeMethodAccessorImpl.j
ava:0 []\n      | MapPartitionsRDD[22] at javaToPython at NativeMethodAccessor
Impl.java:0 []\n      | FileScanRDD[21] at javaToPython at NativeMethodAccesso
rImpl.java:0 []'
Q3 RDD | Top country avg: ('Italy', 405.73) | Time=112.166s
```

QUERY 4

```
45
46 # --- Q4: Count orders per payment method ---
47 t = time.time()
48 q4 = rdd.map(lambda r: (r['payment_method'], 1)) \
49     .reduceByKey(lambda a, b: a + b) \
50     .sortBy(lambda x: -x[1])
51 results['Q4'] = q4.collect()
52 results['Q4_time'] = time.time() - t
53 print("\n=== RDD LINEAGE (Execution Flow) ===")
54 print(q4.toDebugString())
55 print(f"Q4 RDD | Payment methods: {results['Q4']} | Time={results['Q4_time']:.3f}s")
56
```

```
=== RDD LINEAGE (Execution Flow) ===
b'(4) PythonRDD[64] at collect at /home/yousef/spark_project/rdd/queries_rdd.py:
51 []\n | MapPartitionsRDD[63] at mapPartitions at PythonRDD.scala:160 []\n |
ShuffledRDD[62] at partitionBy at NativeMethodAccessorImpl.java:0 []\n +- (4) Pai
rwiseRDD[61] at sortBy at /home/yousef/spark_project/rdd/queries_rdd.py:50 []\n
    | PythonRDD[60] at sortBy at /home/yousef/spark_project/rdd/queries_rdd.py:5
0 []\n      | MapPartitionsRDD[57] at mapPartitions at PythonRDD.scala:160 []\n
        | ShuffledRDD[56] at partitionBy at NativeMethodAccessorImpl.java:0 []\n +
-(4) PairwiseRDD[55] at reduceByKey at /home/yousef/spark_project/rdd/queries_rd
d.py:49 []\n          | PythonRDD[54] at reduceByKey at /home/yousef/spark_proje
ct/rdd/queries_rdd.py:49 []\n              | MapPartitionsRDD[25] at javaToPython at Nat
iveMethodAccessorImpl.java:0 []\n                  | MapPartitionsRDD[24] at javaToPython
at NativeMethodAccessorImpl.java:0 []\n                      | SQLExecutionRDD[23] at javaToPy
thon at NativeMethodAccessorImpl.java:0 []\n                          | MapPartitionsRDD[22] at ja
vaToPython at NativeMethodAccessorImpl.java:0 []\n                              | FileScanRDD[21] at j
avaToPython at NativeMethodAccessorImpl.java:0 []'
Q4 RDD | Payment methods: [('Debit Card', 200805), ('Bank Transfer', 200213), ('
PayPal', 199881), ('Apple Pay', 199814), ('Credit Card', 199410)] | Time=108.196
s
```


QUERY 5

```
57 # --- Q5: Max single order per customer ---
58 t = time.time()
59 q5 = rdd.map(lambda r: (r['customer_id'], float(r['total_price_usd']) if r['total_price_usd']
60     else 0)) \
61     .reduceByKey(lambda a, b: max(a, b)) \
62     .sortBy(lambda x: -x[1])
63 results['Q5'] = q5.take(10)
64 results['Q5_time'] = time.time() - t
65 print("\n=== RDD LINEAGE (Execution Flow) ===")
66 print(q5.toDebugString())
67 print(f"Q5 RDD | Top spender: {results['Q5'][0]} | Time={results['Q5_time']:.3f}s")
```

```
=== RDD LINEAGE (Execution Flow) ===
b'(4) PythonRDD[76] at RDD at PythonRDD.scala:53 []\n | MapPartitionsRDD[74] at
mapPartitions at PythonRDD.scala:160 []\n | ShuffledRDD[73] at partitionBy at
NativeMethodAccessorImpl.java:0 []\n +- (4) PairwiseRDD[72] at sortBy at /home/you
sef/spark_project/rdd/queries_rdd.py:61 []\n | PythonRDD[71] at sortBy at /
home/yousef/spark_project/rdd/queries_rdd.py:61 []\n | MapPartitionsRDD[68]
at mapPartitions at PythonRDD.scala:160 []\n | ShuffledRDD[67] at partitionB
y at NativeMethodAccessorImpl.java:0 []\n +- (4) PairwiseRDD[66] at reduceByKe
y at /home/yousef/spark_project/rdd/queries_rdd.py:60 []\n | PythonRDD[65]
at reduceByKey at /home/yousef/spark_project/rdd/queries_rdd.py:60 []\n
| MapPartitionsRDD[25] at javaToPython at NativeMethodAccessorImpl.java:0 []\n
| MapPartitionsRDD[24] at javaToPython at NativeMethodAccessorImpl.java:0
[]\n | SQLExecutionRDD[23] at javaToPython at NativeMethodAccessorImpl.j
ava:0 []\n | MapPartitionsRDD[22] at javaToPython at NativeMethodAccesso
rImpl.java:0 []\n | FileScanRDD[21] at javaToPython at NativeMethodAccesso
rImpl.java:0 []'
Q5 RDD | Top spender: ('CUS-A9QVX', 2499.75) | Time=183.903s
```

QUERY 6

```
8 # --- Q6: Orders per month ---
9 t = time.time()
10 q6 = rdd.map(lambda r: (str(r['order_date'])[:7] if r['order_date'] else 'Unknown', 1)) \
11         .reduceByKey(lambda a, b: a + b) \
12         .sortBy(lambda x: x[0])
13 results['Q6'] = q6.collect()
14 results['Q6_time'] = time.time() - t
15 print("\n=== RDD LINEAGE (Execution Flow) ===")
16 print(q6.toDebugString())
17 print(f"Q6 RDD | Months: {len(results['Q6'])} | Time={results['Q6_time']:.3f}s")
```

```
=== RDD LINEAGE (Execution Flow) ===
b'(4) PythonRDD[87] at collect at /home/yousef/spark_project/rdd/queries_rdd.py:
73 []\n | MapPartitionsRDD[86] at mapPartitions at PythonRDD.scala:160 []\n |
ShuffledRDD[85] at partitionBy at NativeMethodAccessorImpl.java:0 []\n +-(4) PairwiseRDD[84] at sortBy at /home/yousef/spark_project/rdd/queries_rdd.py:72 []\n
| PythonRDD[83] at sortBy at /home/yousef/spark_project/rdd/queries_rdd.py:7
2 []\n | MapPartitionsRDD[80] at mapPartitions at PythonRDD.scala:160 []\n
| ShuffledRDD[79] at partitionBy at NativeMethodAccessorImpl.java:0 []\n +
-(4) PairwiseRDD[78] at reduceByKey at /home/yousef/spark_project/rdd/queries_rdd.py:71 []\n
| PythonRDD[77] at reduceByKey at /home/yousef/spark_project/rdd/queries_rdd.py:71 []\n
| MapPartitionsRDD[25] at javaToPython at NativeMethodAccessorImpl.java:0 []\n
| MapPartitionsRDD[24] at javaToPython at NativeMethodAccessorImpl.java:0 []\n
| SQLExecutionRDD[23] at javaToPython at NativeMethodAccessorImpl.java:0 []\n
| MapPartitionsRDD[22] at javaToPython at NativeMethodAccessorImpl.java:0 []\n
| FileScanRDD[21] at javaToPython at NativeMethodAccessorImpl.java:0 []'
Q6 RDD | Months: 25 | Time=181.750s
```


QUERY 7

```
0
9 # --- Q7: Revenue by category + country (multi-group) ---
10 t = time.time()
11 q7 = rdd.map(lambda r: ((r['category'], r['country']), float(r['total_price_usd']) if
12     r['total_price_usd'] else 0)) \
13     .reduceByKey(lambda a, b: a + b) \
14     .sortBy(lambda x: -x[1])
15 results['Q7'] = q7.take(10)
16 results['Q7_time'] = time.time() - t
17 print("\n=== RDD LINEAGE (Execution Flow) ===")
18 print(q7.toDebugString())
19 print(f"Q7 RDD | Top group: {results['Q7'][0]} | Time={results['Q7_time']:.3f}s")
```

```
=== RDD LINEAGE (Execution Flow) ===
b'(4) PythonRDD[99] at RDD at PythonRDD.scala:53 []\n | MapPartitionsRDD[97] at
mapPartitions at PythonRDD.scala:160 []\n | ShuffledRDD[96] at partitionBy at
NativeMethodAccessorImpl.java:0 []\n +- (4) PairwiseRDD[95] at sortBy at /home/you
sef/spark_project/rdd/queries_rdd.py:83 []\n | PythonRDD[94] at sortBy at /
home/yousef/spark_project/rdd/queries_rdd.py:83 []\n | MapPartitionsRDD[91]
at mapPartitions at PythonRDD.scala:160 []\n | ShuffledRDD[90] at partitionB
y at NativeMethodAccessorImpl.java:0 []\n +- (4) PairwiseRDD[89] at reduceByKe
y at /home/yousef/spark_project/rdd/queries_rdd.py:82 []\n | PythonRDD[88
] at reduceByKey at /home/yousef/spark_project/rdd/queries_rdd.py:82 []\n
| MapPartitionsRDD[25] at javaToPython at NativeMethodAccessorImpl.java:0 []\n
| MapPartitionsRDD[24] at javaToPython at NativeMethodAccessorImpl.java:0
[]\n | SQLExecutionRDD[23] at javaToPython at NativeMethodAccessorImpl.j
ava:0 []\n | MapPartitionsRDD[22] at javaToPython at NativeMethodAccesso
rImpl.java:0 []\n | FileScanRDD[21] at javaToPython at NativeMethodAccesso
rImpl.java:0 []'
Q7 RDD | Top group: (('Electronics', 'Netherlands'), 14165047.9600000034) | Time=
127.031s
```


QUERY 8

```
90 # --- Q8: Count distinct products per category ---
91 t = time.time()
92 q8 = rdd.map(lambda r: (r['category'], r['product_id'])) \
93     .distinct() \
94     .map(lambda r: (r[0], 1)) \
95     .reduceByKey(lambda a, b: a + b) \
96     .sortBy(lambda x: -x[1])
97 results['Q8'] = q8.collect()
98 results['Q8_time'] = time.time() - t
99 print("\n=== RDD LINEAGE (Execution Flow) ===")
100 print(q8.toDebugString())
101 print(f"Q8 RDD | Distinct products/category: {results['Q8'][:3]} | Time={results['Q8_time']:.3f}s")
102
```

```
=== RDD LINEAGE (Execution Flow) ===
b'(4) PythonRDD[114] at collect at /home/yousef/spark_project/rdd/queries_rdd.py:97 []\n | MapPartitionsRDD[113] at mapPartitions at PythonRDD.scala:160 []\n | ShuffledRDD[112] at partitionBy at NativeMethodAccessorImpl.java:0 []\n +- (4) PairwiseRDD[111] at sortBy at /home/yousef/spark_project/rdd/queries_rdd.py:96 []\n | PythonRDD[110] at sortBy at /home/yousef/spark_project/rdd/queries_rdd.py:96 []\n | MapPartitionsRDD[107] at mapPartitions at PythonRDD.scala:160 []\n | ShuffledRDD[106] at partitionBy at NativeMethodAccessorImpl.java:0 []\n +- (4) PairwiseRDD[105] at reduceByKey at /home/yousef/spark_project/rdd/queries_rdd.py:95 []\n | PythonRDD[104] at reduceByKey at /home/yousef/spark_project/rdd/queries_rdd.py:95 []\n | MapPartitionsRDD[103] at mapPartitions at PythonRDD.scala:160 []\n | ShuffledRDD[102] at partitionBy at NativeMethodAccessorImpl.java:0 []\n +- (4) PairwiseRDD[101] at distinct at /home/yousef/spark_project/rdd/queries_rdd.py:93 []\n | PythonRDD[100] at distinct at /home/yousef/spark_project/rdd/queries_rdd.py:93 []\n | MapPartitionsRDD[25] at javaToPython at NativeMethodAccessorImpl.java:0 []\n | MapPartitionsRDD[24] at javaToPython at NativeMethodAccessorImpl.java:0 []\n | SQLExecutionRDD[23] at javaToPython at NativeMethodAccessorImpl.java:0 []\n | MapPartitionsRDD[22] at javaToPython at NativeMethodAccessorImpl.java:0 []\n | FileScanRDD[21] at javaToPython at NativeMethodAccessorImpl.java:0 []'
Q8 RDD | Distinct products/category: [('Sports', 189088), ('Home', 189059), ('Electronics', 188408)] | Time=136.748s
```

QUERY 9

```
102
103 # --- Q9: Cancelled orders rate per country ---
104 t = time.time()
105 total_by_country = rdd.map(lambda r: (r['country'], 1)).reduceByKey(lambda a,b: a+b)
106 cancelled = rdd.filter(lambda r: r['order_status'] == 'Cancelled') \
107     .map(lambda r: (r['country'], 1)).reduceByKey(lambda a,b: a+b)
108 q9 = cancelled.join(total_by_country) \
109     .mapValues(lambda x: round(x[0]/x[1]*100, 2)) \
110     .sortBy(lambda x: -x[1])
111 results['Q9'] = q9.take(10)
112 results['Q9_time'] = time.time() - t
113 print("\n=== RDD LINEAGE (Execution Flow) ===")
114 print(q9.toDebugString())
115 print(f"Q9 RDD | Top cancelled rate: {results['Q9'][0]} | Time={results['Q9_time']:.3f}s")
```

```
=== RDD LINEAGE (Execution Flow) ===
b'(4) PythonRDD[134] at RDD at PythonRDD.scala:53 []\n | MapPartitionsRDD[131] at mapPartitions at PythonRDD.scala:160 []\n | ShuffledRDD[130] at partitionBy at <unknown>:0 []\n +- (4) PairwiseRDD[129] at sortBy at /home/yousef/spark_project/rdd/queries_rdd.py:110 []\n | PythonRDD[128] at sortBy at /home/yousef/spark_project/rdd/queries_rdd.py:110 []\n | PartitionerAwareUnionRDD[125] at union at NativeMethodAccessorImpl.java:0 []\n | PythonRDD[123] at RDD at PythonRDD.scala:53 []\n | MapPartitionsRDD[122] at mapPartitions at PythonRDD.scala:160 []\n | ShuffledRDD[121] at partitionBy at <unknown>:0 []\n +- (4) PairwiseRDD[120] at reduceByKey at /home/yousef/spark_project/rdd/queries_rdd.py:107 []\n | PythonRDD[119] at reduceByKey at /home/yousef/spark_project/rdd/queries_rdd.py:107 []\n | MapPartitionsRDD[25] at javaToPython at NativeMethodAccessorImpl.java:0 []\n | MapPartitionsRDD[24] at javaToPython at NativeMethodAccessorImpl.java:0 []\n | SQLExecutionRDD[23] at javaToPython at NativeMethodAccessorImpl.java:0 []\n | MapPartitionsRDD[22] at javaToPython at NativeMethodAccessorImpl.java:0 []\n | FileScanRDD[21] at javaToPython at NativeMethodAccessorImpl.java:0 []\n | PythonRDD[124] at RDD at PythonRDD.scala:53 []\n | MapPartitionsRDD[118] at mapPartitions at PythonRDD.scala:160 []\n | ShuffledRDD[117] at partitionBy at NativeMethodAccessorImpl.java:0 []\n +- (4) PairwiseRDD[116] at reduceByKey at /home/yousef/spark_project/rdd/queries_rdd.py:105 []\n | PythonRDD[115] at reduceByKey at /home/yousef/spark_project/rdd/queries_rdd.py:105 []\n | MapPartitionsRDD[25] at javaToPython at NativeMethodAccessorImpl.java:0 []\n | MapPartitionsRDD[24] at javaToPython at NativeMethodAccessorImpl.java:0 []\n | SQLExecutionRDD[23] at javaToPython at NativeMethodAccessorImpl.java:0 []\n | MapPartitionsRDD[22] at javaToPython at NativeMethodAccessorImpl.java:0 []\n | FileScanRDD[21] at javaToPython at NativeMethodAccessorImpl.java:0 []'
Q9 RDD | Top cancelled rate: ('Spain', 5.14) | Time=253.560s
```


QUERY 10

```
116
117 # --- Q10: High-value customers (total spend > 5000) ---
118 t = time.time()
119 q10 = rdd.map(lambda r: (r['customer_id'], float(r['total_price_usd']) if r['total_price_usd']
120     else 0)) \
121     .reduceByKey(lambda a, b: a + b) \
122     .filter(lambda x: x[1] > 5000) \
123     .sortBy(lambda x: -x[1])
124 results['Q10_count'] = q10.count()
125 results['Q10_time'] = time.time() - t
126 print("\n=== RDD LINEAGE (Execution Flow) ===")
127 print(q10.toDebugString())
128 print(f"Q10 RDD | High-value customers: {results['Q10_count']} | Time={results['Q10_time']:.3f}
129     s")
130
```

=== RDD LINEAGE (Execution Flow) ===

b'(4) PythonRDD[141] at RDD at PythonRDD.scala:53 []\n | MapPartitionsRDD[138] at mapPartitions at PythonRDD.scala:160 []\n | ShuffledRDD[137] at partitionBy at <unknown>:0 []\n +- (4) PairwiseRDD[136] at reduceByKey at /home/yousef/spark_project/rdd/queries_rdd.py:120 []\n | PythonRDD[135] at reduceByKey at /home/yousef/spark_project/rdd/queries_rdd.py:120 []\n | MapPartitionsRDD[25] at javaToPython at NativeMethodAccessorImpl.java:0 []\n | MapPartitionsRDD[24] at javaToPython at NativeMethodAccessorImpl.java:0 []\n | SQLExecutionRDD[23] at javaToPython at NativeMethodAccessorImpl.java:0 []\n | MapPartitionsRDD[22] at javaToPython at NativeMethodAccessorImpl.java:0 []\n | FileScanRDD[21] at javaToPython at NativeMethodAccessorImpl.java:0 []'

Q10 RDD | High-value customers: 0 | Time=145.656s

All RDD timings saved to results/rdd_timings.json

Activate Windows

Go to Settings to activate Windows.

SHUFFLE SIZE AND MEMORY USAGE

DataFrame RESULTS

19	showString at NativeMethodAccessorImpl.java:0	+details	(kill)	2026/04/24 19:33:56	5 s	0/4 (1 running)			
Page:	1					1 Pages. Jump to	1	Show	100 items in a page. Go
▼ Completed Stages (14)									
Page:	1					1 Pages. Jump to	1	Show	100 items in a page. Go
Stage Id ▼	Description		Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
18	showString at NativeMethodAccessorImpl.java:0	+details	2026/04/24 19:33:53	2 s	1/1			14.6 MiB	
16	showString at NativeMethodAccessorImpl.java:0	+details	2026/04/24 19:33:13	39 s	4/4	391.6 MiB			14.6 MiB
15	showString at NativeMethodAccessorImpl.java:0	+details	2026/04/24 19:33:12	0.2 s	1/1			1590.0 B	
13	showString at NativeMethodAccessorImpl.java:0	+details	2026/04/24 19:32:54	17 s	4/4	391.6 MiB			1590.0 B
12	showString at NativeMethodAccessorImpl.java:0	+details	2026/04/24 19:32:52	0.2 s	1/1			2.3 KiB	
10	showString at NativeMethodAccessorImpl.java:0	+details	2026/04/24 19:32:33	19 s	4/4	391.6 MiB			2.3 KiB
9	showString at NativeMethodAccessorImpl.java:0	+details	2026/04/24 19:32:30	0.6 s	1/1			1144.0 B	
7	showString at NativeMethodAccessorImpl.java:0	+details	2026/04/24 19:31:49	39 s	4/4	391.6 MiB			1144.0 B
6	showString at NativeMethodAccessorImpl.java:0	+details	2026/04/24 19:31:41	2 s	1/1	64.0 KiB			
5	showString at NativeMethodAccessorImpl.java:0	+details	2026/04/24 18:09:45	0.9 s	1/1	64.0 KiB			
4	count at NativeMethodAccessorImpl.java:0	+details	2026/04/24 18:09:43	0.3 s	1/1			236.0 B	
2	count at NativeMethodAccessorImpl.java:0	+details	2026/04/24 18:09:38	4 s	4/4	391.6 MiB			236.0 B
1	csv at NativeMethodAccessorImpl.java:0	+details	2026/04/24 18:09:09	26 s	4/4	391.6 MiB			
0	csv at NativeMethodAccessorImpl.java:0	+details	2026/04/24 18:09:06	1 s	1/1	64.0 KiB			
Page:	1					1 Pages. Jump to	1	Show	100 items in a page. Go



Jobs

Stages

Storage

Environment

Executors

SQL / DataFrame

EcommerceAnalysis application UI

Files

Spark Jobs (?)

User: yousef

Total Uptime: 2.0 min

Scheduling Mode: FIFO

Active Jobs: 1

Completed Jobs: 14

Event Timeline

Active Jobs (1)

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

Job Id ▾	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
14	showString at NativeMethodAccessorImpl.java:0 showString at NativeMethodAccessorImpl.java:0	2026/04/24 16:47:11 (kill)	4 s	0/1	0/4

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

Completed Jobs (14)

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

Job Id ▾	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
13	showString at NativeMethodAccessorImpl.java:0 showString at NativeMethodAccessorImpl.java:0	2026/04/24 16:47:07	4 s	1/1 (1 skipped)	1/1 (4 skipped)
12	showString at NativeMethodAccessorImpl.java:0	2026/04/24 16:46:51	16 s	1/1	4/4

QUERY 1


```

9
10 spark = get_spark()
11 df = load_data(spark)
12 timings = {}
13
14 # --- Q1: Filter: Completed orders > $500 ---
15 t = time.time()
16 q1 = df.filter((F.col("order_status") == "Completed") & (F.col("total_price_usd") > 500))
17 print(f"\nQ1 DataFrame | Count={q1.count()}")
18 print("=== Q1 EXPLAIN ==="); q1.explain(True)
19 timings['Q1_time'] = time.time() - t

```

```

Q1 DataFrame | Count=198151
=== Q1 EXPLAIN ===
== Parsed Logical Plan ==
FileScan [r (('order_status = Completed) AND ('total_price_usd > 500))
  Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_
weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,cu
stomer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more f
ields] csv

== Analyzed Logical Plan ==
order_id: string, order_date: timestamp, order_year: int, order_month: int, order_day: int, order_hour: int, order_minute: int, orde
r_second: int, is_weekend: string, order_status: string, return_reason: string, customer_id: string, customer_name: string, gender:
string, age: int, customer_segment: string, country: string, city: string, customer_loyalty_score: double, total_orders_by_customer:
int, account_creation_date: date, product_id: string, product_name: string, category: string, ... 38 more fields
Filter (((order_status#26 = Completed) AND (total_price_usd#50 > cast(500 as double))))
+- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_we
ekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,cu
stomer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more f
ields] csv

== Optimized Logical Plan ==
Filter (((isnotnull(order_status#26) AND isnotnull(total_price_usd#50)) AND ((order_status#26 = Completed) AND (total_price_usd#50 >
500.0))))
+- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_we
ekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,cu
stomer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more f
ields] csv

== Physical Plan ==
*(1) Filter (((isnotnull(order_status#26) AND isnotnull(total_price_usd#50)) AND (order_status#26 = Completed)) AND (total_price_usd
#50 > 500.0))
+- FileScan csv [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,i
s_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#3
4,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 mo
re fields] Batched: false, DataFilters: [isnotnull(order_status#26), isnotnull(total_price_usd#50), (order_status#26 = Completed), (
total..., Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/yousef/spark_project/data/ecommerce.csv], PartitionFilters: [
], PushedFilters: [IsNotNull(order_status), IsNotNull(total_price_usd), EqualTo(order_status,Completed), GreaterTha..., ReadSchema:
struct<order_id:string,order_date:timestamp,order_year:int,order_month:int,order_day:int,order_ho... activate Windows.

```

```

== Physical Plan ==
*(1) Filter (((isnotnull(order_status#26) AND isnotnull(total_price_usd#50)) AND (order_status#26 = Completed)) AND (total_price_usd
#50 > 500.0))
+- FileScan csv [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,i
s_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#3
4,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 mo
re fields] Batched: false, DataFilters: [isnotnull(order_status#26), isnotnull(total_price_usd#50), (order_status#26 = Completed), (
total..., Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/yousef/spark_project/data/ecommerce.csv], PartitionFilters: [
], PushedFilters: [IsNotNull(order_status), IsNotNull(total_price_usd), EqualTo(order_status,Completed), GreaterTha..., ReadSchema:
struct<order_id:string,order_date:timestamp,order_year:int,order_month:int,order_day:int,order_ho...

+-----+-----+
| category|total_revenue|
+-----+-----+
|Electronics|1.3986598694E8|
|   Home   |8.782867776E7|
|   Sports  |7.291747085E7|
|   Health  |5.746792555E7|
|   Clothing|4.52057959E7|
+-----+-----+

```

QUERY 2


```

57
58 # --- Q2: Aggregation ---
59 t = time.time()
60 q2 = df.groupby("category") \
61     .agg(F.round(F.sum("total_price_usd"), 2).alias("total_revenue")) \
62     .orderBy(F.desc("total_revenue"))
63 run_query("Q2 - Revenue by category", q2)
64 timings['Q2_time'] = time.time() - t

```

```
=====
```

```
Q2 - Revenue by category RESULT
```

```
=====
```

```
+-----+-----+
```

```
| category| total_revenue|
```

```
+-----+-----+
```

```
|Electronics|1.3986598694E8|
```

```
|      Home| 8.782867776E7|
```

```
|    Sports| 7.291747085E7|
```

```
|    Health| 5.746792555E7|
```

```
|  Clothing| 4.52057959E7|
```

```
+-----+-----+
```

```
.....
Q2 - Revenue by category EXPLAIN PLAN
.....
```

```
== Parsed Logical Plan ==
```

```
'Sort ['total_revenue DESC NULLS LAST], true
```

```
+-- Aggregate [category#40], [category#40, round(sum(total_price_usd#50), 2) AS total_revenue#894]
```

```
  +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

```
== Analyzed Logical Plan ==
```

```
category: string, total_revenue: double
```

```
Sort [total_revenue#894 DESC NULLS LAST], true
```

```
+-- Aggregate [category#40], [category#40, round(sum(total_price_usd#50), 2) AS total_revenue#894]
```

```
  +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

```
== Optimized Logical Plan ==
```

```
Sort [total_revenue#894 DESC NULLS LAST], true
```

```
+-- Aggregate [category#40], [category#40, round(sum(total_price_usd#50), 2) AS total_revenue#894]
```

```
  +- Project [category#40, total_price_usd#50]
```

```
    +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

```
== Physical Plan ==
```

```
AdaptiveSparkPlan isFinalPlan=false
```

```
+-- Sort [total_revenue#894 DESC NULLS LAST], true, 0
```

```
  +- Exchange rangepartitioning(total_revenue#894 DESC NULLS LAST, 8), ENSURE_REQUIREMENTS, [plan_id=152]
```

```
    +- HashAggregate(keys=[category#40], functions=[sum(total_price_usd#50)], output=[category#40, total_revenue#894])
```

```
      +- Exchange hashpartitioning(category#40, 8), ENSURE_REQUIREMENTS, [plan_id=149]
```

```
        +- HashAggregate(keys=[category#40], functions=[partial_sum(total_price_usd#50)], output=[category#40, sum#904])
```

```
          +- FileScan csv [category#40,total_price_usd#50] Batched: false, DataFilters: [], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/yousef/spark_project/data/ecommerce.csv], PartitionFilters: [], PushedFilters: [], ReadSchema: struct<category:string,total_price_usd:double>
```

QUERY 3

```
46
47 # --- Q3: Average by country ---
48 t = time.time()
49 q3 = df.groupby("country") \
50     .agg(F.round(F.avg("total_price_usd"), 2).alias("avg_order_value")) \
51     .orderBy(F.desc("avg_order_value"))
52 run_query("Q3 - Avg order value by country", q3)
53 timings['Q3_time'] = time.time() - t
54
```

```
=====
Q3 - Avg order value by country RESULT
=====
+-----+-----+
|      country|avg_order_value|
+-----+-----+
|      Italy|      405.73|
| Australia|      404.23|
|      Spain|      403.61|
| United States|      403.41|
| Netherlands|      403.24|
| United Kingdom|      403.15|
|      Belgium|      402.73|
|      Germany|      402.64|
|      Canada|      401.94|
|      France|      401.71|
+-----+-----+
```



```
.....
Q3 - Avg order value by country EXPLAIN PLAN
.....
```

```
== Parsed Logical Plan ==
```

```
'Sort ['avg_order_value DESC NULLS LAST], true
```

```
+-- Aggregate [country#33], [country#33, round(avg(total_price_usd#50), 2) AS avg_order_value#975]
```

```
  +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

```
== Analyzed Logical Plan ==
```

```
country: string, avg_order_value: double
```

```
Sort [avg_order_value#975 DESC NULLS LAST], true
```

```
+-- Aggregate [country#33], [country#33, round(avg(total_price_usd#50), 2) AS avg_order_value#975]
```

```
  +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

```
== Optimized Logical Plan ==
```

```
Sort [avg_order_value#975 DESC NULLS LAST], true
```

```
+-- Aggregate [country#33], [country#33, round(avg(total_price_usd#50), 2) AS avg_order_value#975]
```

```
  +- Project [country#33, total_price_usd#50]
```

```
    +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

```
== Physical Plan ==
```

```
AdaptiveSparkPlan isFinalPlan=false
```

```
+-- Sort [avg_order_value#975 DESC NULLS LAST], true, 0
```

```
  +- Exchange rangepartitioning(avg_order_value#975 DESC NULLS LAST, 8), ENSURE_REQUIREMENTS, [plan_id=217]
```

```
    +- HashAggregate(keys=[country#33], functions=[avg(total_price_usd#50)], output=[country#33, avg_order_value#975])
```

```
      +- Exchange hashpartitioning(country#33, 8), ENSURE_REQUIREMENTS, [plan_id=214]
```

```
        +- HashAggregate(keys=[country#33], functions=[partial_avg(total_price_usd#50)], output=[country#33, sum#986, count#987L])
```

```
          +- FileScan csv [country#33,total_price_usd#50] Batched: false, DataFilters: [], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/yousef/spark_project/data/ecommerce.csv], PartitionFilters: [], PushedFilters: [], ReadSchema: struct<country:string,total_price_usd:double>
```

QUERY 4

```
55
56 # --- Q4: Count by payment method ---
57 t = time.time()
58 q4 = df.groupby("payment_method") \
59     .agg(F.count("*").alias("order_count")) \
60     .orderBy(F.desc("order_count"))
61 run_query("Q4 - Orders by payment method", q4)
62 timings['Q4_time'] = time.time() - t
63
```

```
=====
Q4 - Orders by payment method RESULT
=====
+-----+-----+
|payment_method|order_count|
+-----+-----+
| Debit Card   | 200805    |
| Bank Transfer| 200213    |
| PayPal       | 199881    |
| Apple Pay    | 199814    |
| Credit Card  | 199410    |
+-----+-----+
```



```
-----  
Q4 - Orders by payment method EXPLAIN PLAN  
-----
```

```
== Parsed Logical Plan ==
```

```
'Sort ['order_count DESC NULLS LAST], true
```

```
+-- Aggregate [payment_method#55], [payment_method#55, count(1) AS order_count#1060L]
```

```
  +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

```
== Analyzed Logical Plan ==
```

```
payment_method: string, order_count: bigint
```

```
Sort [order_count#1060L DESC NULLS LAST], true
```

```
+-- Aggregate [payment_method#55], [payment_method#55, count(1) AS order_count#1060L]
```

```
  +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

```
== Optimized Logical Plan ==
```

```
Sort [order_count#1060L DESC NULLS LAST], true
```

```
+-- Aggregate [payment_method#55], [payment_method#55, count(1) AS order_count#1060L]
```

```
  +- Project [payment_method#55]
```

```
    +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

```
== Physical Plan ==
```

```
AdaptiveSparkPlan isFinalPlan=false
```

```
+-- Sort [order_count#1060L DESC NULLS LAST], true, 0
```

```
  +- Exchange rangepartitioning(order_count#1060L DESC NULLS LAST, 8), ENSURE_REQUIREMENTS, [plan_id=282]
```

```
    +- HashAggregate(keys=[payment_method#55], functions=[count(1)], output=[payment_method#55, order_count#1060L])
```

```
      +- Exchange hashpartitioning(payment_method#55, 8), ENSURE_REQUIREMENTS, [plan_id=279]
```

```
        +- HashAggregate(keys=[payment_method#55], functions=[partial_count(1)], output=[payment_method#55, count#1070L])
```

```
          +- FileScan csv [payment_method#55] Batched: false, DataFilters: [], Format: CSV, Location: InMemory  
FileIndex(1 paths)[file:/home/yousef/spark_project/data/ecommerce.csv], PartitionFilters: [], PushedFilters: [], ReadSchema: struct<payment_method:string>
```

QUERY 5

```
64
65 # --- Q5: Max order per customer ---
66 t = time.time()
67 q5 = df.groupby("customer_id") \
68     .agg(F.max("total_price_usd").alias("max_order")) \
69     .orderBy(F.desc("max_order"))
70 run_query("Q5 - Max order per customer", q5)
71 timings['Q5_time'] = time.time() - t
72
```

```
=====
Q5 - Max order per customer RESULT
=====
+-----+-----+
|customer_id|max_order|
+-----+-----+
| CUS-A9QVX| 2499.75|
| CUS-SF2Q8| 2499.45|
| CUS-WKXXV| 2499.2|
| CUS-LY7WI| 2499.15|
| CUS-ESEP9| 2498.95|
| CUS-8TUG3| 2498.8|
| CUS-F3TH0| 2498.75|
| CUS-UR59J| 2498.45|
| CUS-9SBVR| 2497.8|
| CUS-1BE0M| 2497.6|
+-----+-----+
only showing top 10 rows
```



```
-----  
Q5 - Max order per customer EXPLAIN PLAN  
-----
```

```
== Parsed Logical Plan ==
```

```
'Sort ['max_order DESC NULLS LAST], true
```

```
+-- Aggregate [customer_id#28], [customer_id#28, max(total_price_usd#50) AS max_order#1138]
```

```
  +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

```
== Analyzed Logical Plan ==
```

```
customer_id: string, max_order: double
```

```
Sort [max_order#1138 DESC NULLS LAST], true
```

```
+-- Aggregate [customer_id#28], [customer_id#28, max(total_price_usd#50) AS max_order#1138]
```

```
  +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

```
== Optimized Logical Plan ==
```

```
Sort [max_order#1138 DESC NULLS LAST], true
```

```
+-- Aggregate [customer_id#28], [customer_id#28, max(total_price_usd#50) AS max_order#1138]
```

```
  +- Project [customer_id#28, total_price_usd#50]
```

```
    +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

```
== Physical Plan ==
```

```
AdaptiveSparkPlan isFinalPlan=false
```

```
+-- Sort [max_order#1138 DESC NULLS LAST], true, 0
```

```
  +- Exchange rangepartitioning(max_order#1138 DESC NULLS LAST, 8), ENSURE_REQUIREMENTS, [plan_id=347]
```

```
    +- HashAggregate(keys=[customer_id#28], functions=[max(total_price_usd#50)], output=[customer_id#28, max_order#1138])
```

```
      +- Exchange hashpartitioning(customer_id#28, 8), ENSURE_REQUIREMENTS, [plan_id=344]
```

```
        +- HashAggregate(keys=[customer_id#28], functions=[partial_max(total_price_usd#50)], output=[customer_id#28, max#1148])
```

```
          +- FileScan csv [customer_id#28,total_price_usd#50] Batched: false, DataFilters: [], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/yousef/spark_project/data/ecommerce.csv], PartitionFilters: [], PushedFilters: [], ReadSchema: struct<customer_id:string,total_price_usd:double>
```

QUERY 6

```
73
74 # --- Q6: Orders per month ---
75 t = time.time()
76 q6 = df.withColumn("month", F.date_format(F.col("order_date"), "yyyy-MM")) \
77     .groupBy("month") \
78     .agg(F.count("**").alias("order_count")) \
79     .orderBy("month")
80 run_query("Q6 - Orders per month", q6, show_n=24)
81 timings['Q6_time'] = time.time() - t
82
```

```
=====
Q6 - Orders per month RESULT
=====
+-----+-----+
| month|order_count|
+-----+-----+
|2024-02|      37052|
|2024-03|      42716|
|2024-04|      41415|
|2024-05|      42697|
|2024-06|      40756|
|2024-07|      42255|
|2024-08|      42241|
|2024-09|      41035|
|2024-10|      42245|
|2024-11|      40800|
|2024-12|      42037|
|2025-01|      42536|
|2025-02|      38349|
|2025-03|      42363|
|2025-04|      41021|
|2025-05|      42403|
|2025-06|      40943|
|2025-07|      42774|
|2025-08|      42558|
|2025-09|      40970|
|2025-10|      42603|
|2025-11|      40931|
|2025-12|      42575|
|2026-01|      42564|
+-----+-----+
```

Activate Win

Q6 - Orders per month EXPLAIN PLAN

== Parsed Logical Plan ==

```
'Sort [month ASC NULLS FIRST], true
+- Aggregate [month#1156], [month#1156, count(1) AS order_count#1284L]
  +- Project [order_id#17, order_date#18, order_year#19, order_month#20, order_day#21, order_hour#22, order_minute#23, order_second#24, is_weekend#25, order_status#26, return_reason#27, customer_id#28, customer_name#29, gender#30, age#31, customer_segment#32, country#33, city#34, customer_loyalty_score#35, total_orders_by_customer#36, account_creation_date#37, product_id#38, product_name#39, category#40, ... 39 more fields]
    +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

== Analyzed Logical Plan ==

```
month: string, order_count: bigint
Sort [month#1156 ASC NULLS FIRST], true
+- Aggregate [month#1156], [month#1156, count(1) AS order_count#1284L]
  +- Project [order_id#17, order_date#18, order_year#19, order_month#20, order_day#21, order_hour#22, order_minute#23, order_second#24, is_weekend#25, order_status#26, return_reason#27, customer_id#28, customer_name#29, gender#30, age#31, customer_segment#32, country#33, city#34, customer_loyalty_score#35, total_orders_by_customer#36, account_creation_date#37, product_id#38, product_name#39, category#40, ... 39 more fields]
    +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

== Optimized Logical Plan ==

```
Sort [month#1156 ASC NULLS FIRST], true
+- Aggregate [month#1156], [month#1156, count(1) AS order_count#1284L]
  +- Project [date_format(order_date#18, yyyy-MM, Some(Africa/Cairo)) AS month#1156]
    +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

== Physical Plan ==

AdaptiveSparkPlan isFinalPlan=false

```
+- Sort [month#1156 ASC NULLS FIRST], true, 0
  +- Exchange rangepartitioning(month#1156 ASC NULLS FIRST, 8), ENSURE_REQUIREMENTS, [plan_id=421]
    +- HashAggregate(keys=[month#1156], functions=[count(1)], output=[month#1156, order_count#1284L])
      +- Exchange hashpartitioning(month#1156, 8), ENSURE_REQUIREMENTS, [plan_id=418]
        +- HashAggregate(keys=[month#1156], functions=[partial_count(1)], output=[month#1156, count#1294L])
          +- Project [date_format(order_date#18, yyyy-MM, Some(Africa/Cairo)) AS month#1156]
            +- FileScan csv [order_date#18] Batched: false, DataFilters: [], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/yousef/spark_project/data/ecommerce.csv], PartitionFilters: [], PushedFilters: [], ReadSchema: struct<order_date:timestamp>
```


QUERY 7

```
83
84 # --- Q7: Multi-group aggregation ---
85 t = time.time()
86 q7 = df.groupby("category", "country") \
87     .agg(F.round(F.sum("total_price_usd"), 2).alias("revenue")) \
88     .orderBy(F.desc("revenue"))
89 run_query("Q7 - Revenue by category & country", q7)
90 timings['Q7_time'] = time.time() - t
91
```

```
=====
Q7 - Revenue by category & country RESULT
=====
+-----+-----+-----+
| category| country| revenue|
+-----+-----+-----+
|Electronics| Netherlands|1.416504796E7|
|Electronics| Italy|1.410976446E7|
|Electronics| Belgium|1.406366308E7|
|Electronics| Spain|1.400882483E7|
|Electronics| United Kingdom|1.400408099E7|
|Electronics| Canada|1.397057633E7|
|Electronics| United States|1.396531785E7|
|Electronics| Australia|1.395247316E7|
|Electronics| Germany|1.387607852E7|
|Electronics| France|1.375015976E7|
+-----+-----+-----+
only showing top 10 rows
```

```
-----  
Q7 - Revenue by category & country EXPLAIN PLAN  
-----
```

```
== Parsed Logical Plan ==
```

```
'Sort ['revenue DESC NULLS LAST], true
```

```
+-- Aggregate [category#40, country#33], [category#40, country#33, round(sum(total_price_usd#50), 2) AS revenue#1362]  
]
```

```
+-- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

```
== Analyzed Logical Plan ==
```

```
category: string, country: string, revenue: double
```

```
Sort [revenue#1362 DESC NULLS LAST], true
```

```
+-- Aggregate [category#40, country#33], [category#40, country#33, round(sum(total_price_usd#50), 2) AS revenue#1362]  
]
```

```
+-- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

```
== Optimized Logical Plan ==
```

```
Sort [revenue#1362 DESC NULLS LAST], true
```

```
+-- Aggregate [category#40, country#33], [category#40, country#33, round(sum(total_price_usd#50), 2) AS revenue#1362]  
]
```

```
+-- Project [country#33, category#40, total_price_usd#50]
```

```
+-- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

```
== Physical Plan ==
```

```
AdaptiveSparkPlan isFinalPlan=false
```

```
+-- Sort [revenue#1362 DESC NULLS LAST], true, 0
```

```
+-- Exchange rangepartitioning(revenue#1362 DESC NULLS LAST, 8), ENSURE_REQUIREMENTS, [plan_id=486]
```

```
+-- HashAggregate(keys=[category#40, country#33], functions=[sum(total_price_usd#50)], output=[category#40, country#33, revenue#1362])
```

```
+-- Exchange hashpartitioning(category#40, country#33, 8), ENSURE_REQUIREMENTS, [plan_id=483]
```

```
+-- HashAggregate(keys=[category#40, country#33], functions=[partial_sum(total_price_usd#50)], output=[category#40, country#33, sum#1376])
```

```
+-- FileScan csv [country#33,category#40,total_price_usd#50] Batched: false, DataFilters: [], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/yousef/spark_project/data/ecommerce.csv], PartitionFilters: [], PushedFilters: [], ReadSchema: struct<country:string,category:string,total_price_usd:double>
```

QUERY 8

```
92
93 # --- Q8: Distinct count ---
94 t = time.time()
95 q8 = df.groupBy("category") \
96     .agg(F.countDistinct("product_id").alias("distinct_products")) \
97     .orderBy(F.desc("distinct_products"))
98 run_query("Q8 - Distinct products per category", q8)
99 timings['Q8_time'] = time.time() - t
100
```

```
=====
Q8 - Distinct products per category RESULT
=====
+-----+-----+
| category|distinct_products|
+-----+-----+
| Sports|189088|
| Home|189059|
| Electronics|188408|
| Clothing|188183|
| Health|188127|
+-----+-----+
```



```
.....
Q8 - Distinct products per category EXPLAIN PLAN
.....
```

```
== Parsed Logical Plan ==
```

```
'Sort ['distinct_products DESC NULLS LAST], true
```

```
+· Aggregate [category#40], [category#40, count(distinct product_id#38) AS distinct_products#1448L]
```

```
+· Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

```
== Analyzed Logical Plan ==
```

```
category: string, distinct_products: bigint
```

```
'Sort [distinct_products#1448L DESC NULLS LAST], true
```

```
+· Aggregate [category#40], [category#40, count(distinct product_id#38) AS distinct_products#1448L]
```

```
+· Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

```
== Optimized Logical Plan ==
```

```
Sort [distinct_products#1448L DESC NULLS LAST], true
```

```
+· Aggregate [category#40], [category#40, count(distinct product_id#38) AS distinct_products#1448L]
```

```
+· Project [product_id#38, category#40]
```

```
+· Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

```
== Physical Plan ==
```

```
AdaptiveSparkPlan isFinalPlan=false
```

```
+· Sort [distinct_products#1448L DESC NULLS LAST], true, 0
```

```
+· Exchange rangepartitioning(distinct_products#1448L DESC NULLS LAST, 8), ENSURE_REQUIREMENTS, [plan_id=617]
```

```
+· HashAggregate(keys=[category#40], functions=[count(distinct product_id#38)], output=[category#40, distinct_products#1448L])
```

```
+· Exchange hashpartitioning(category#40, 8), ENSURE_REQUIREMENTS, [plan_id=614]
```

```
+· HashAggregate(keys=[category#40], functions=[partial_count(distinct product_id#38)], output=[category#40, count#1460L])
```

```
+· HashAggregate(keys=[category#40, product_id#38], functions=[], output=[category#40, product_id#38])
```

```
+· Exchange hashpartitioning(category#40, product_id#38, 8), ENSURE_REQUIREMENTS, [plan_id=610]
```

```
+· HashAggregate(keys=[category#40, product_id#38], functions=[], output=[category#40, product_id#38])
```

```
+· FileScan csv [product_id#38,category#40] Batched: false, DataFilters: [], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/yousef/spark_project/data/ecommerce.csv], PartitionFilters: [], PushedFilters: [], ReadSchema: struct<product_id:string,category:string>
```

Activate Windows

QUERY 9

```
101
102 # --- Q9: Window function ---
103 t = time.time()
104 w = Window.orderBy("order_date").rowsBetween(Window.unboundedPreceding, Window.currentRow)
105 q9 = df.withColumn("cumulative_revenue", F.sum("total_price_usd").over(w)) \
106     .select("order_date", "total_price_usd", "cumulative_revenue")
107 run_query("Q9 - Cumulative revenue (window)", q9)
108 timings['Q9_time'] = time.time() - t
109
```

=====

Q9 - Cumulative revenue (window) RESULT

=====

order_date	total_price_usd	cumulative_revenue
2024-02-03 04:31:...	496.32	496.32
2024-02-03 04:33:...	92.38	588.7
2024-02-03 04:34:...	1191.02	1779.72
2024-02-03 04:34:...	197.17	1976.89
2024-02-03 04:35:...	53.1	2029.99
2024-02-03 04:36:...	471.62	2501.61
2024-02-03 04:37:...	308.22	2809.83
2024-02-03 04:37:...	91.47	2901.2999999999997
2024-02-03 04:37:...	422.65	3323.95
2024-02-03 04:40:...	400.22	3724.17

only showing top 10 rows

```
.....
Q9 - Cumulative revenue (window) EXPLAIN PLAN
.....
```

```
== Parsed Logical Plan ==
```

```
'Project ['order_date', 'total_price_usd', 'cumulative_revenue]
```

```
+-- Project [order_id#17, order_date#18, order_year#19, order_month#20, order_day#21, order_hour#22, order_minute#23, order_second#24, is_weekend#25, order_status#26, return_reason#27, customer_id#28, customer_name#29, gender#30, age#31, customer_segment#32, country#33, city#34, customer_loyalty_score#35, total_orders_by_customer#36, account_creation_date#37, product_id#38, product_name#39, category#40, ... 39 more fields]
```

```
+-- Project [order_id#17, order_date#18, order_year#19, order_month#20, order_day#21, order_hour#22, order_minute#23, order_second#24, is_weekend#25, order_status#26, return_reason#27, customer_id#28, customer_name#29, gender#30, age#31, customer_segment#32, country#33, city#34, customer_loyalty_score#35, total_orders_by_customer#36, account_creation_date#37, product_id#38, product_name#39, category#40, ... 40 more fields]
```

```
+-- Window [sum(total_price_usd#50) windowSpecDefinition(order_date#18 ASC NULLS FIRST, specifiedWindowFrame(RowFrame, unboundedPreceding$, currentRow$())) AS cumulative_revenue#1473], [order_date#18 ASC NULLS FIRST]
```

```
+-- Project [order_id#17, order_date#18, order_year#19, order_month#20, order_day#21, order_hour#22, order_minute#23, order_second#24, is_weekend#25, order_status#26, return_reason#27, customer_id#28, customer_name#29, gender#30, age#31, customer_segment#32, country#33, city#34, customer_loyalty_score#35, total_orders_by_customer#36, account_creation_date#37, product_id#38, product_name#39, category#40, ... 38 more fields]
```

```
+-- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

```
== Analyzed Logical Plan ==
```

```
order_date: timestamp, total_price_usd: double, cumulative_revenue: double
```

```
Project [order_date#18, total_price_usd#50, cumulative_revenue#1473]
```

```
+-- Project [order_id#17, order_date#18, order_year#19, order_month#20, order_day#21, order_hour#22, order_minute#23, order_second#24, is_weekend#25, order_status#26, return_reason#27, customer_id#28, customer_name#29, gender#30, age#31, customer_segment#32, country#33, city#34, customer_loyalty_score#35, total_orders_by_customer#36, account_creation_date#37, product_id#38, product_name#39, category#40, ... 39 more fields]
```

```
+-- Project [order_id#17, order_date#18, order_year#19, order_month#20, order_day#21, order_hour#22, order_minute#23, order_second#24, is_weekend#25, order_status#26, return_reason#27, customer_id#28, customer_name#29, gender#30, age#31, customer_segment#32, country#33, city#34, customer_loyalty_score#35, total_orders_by_customer#36, account_creation_date#37, product_id#38, product_name#39, category#40, ... 40 more fields]
```

```
+-- Window [sum(total_price_usd#50) windowSpecDefinition(order_date#18 ASC NULLS FIRST, specifiedWindowFrame(RowFrame, unboundedPreceding$, currentRow$())) AS cumulative_revenue#1473], [order_date#18 ASC NULLS FIRST]
```

```
+-- Project [order_id#17, order_date#18, order_year#19, order_month#20, order_day#21, order_hour#22, order_minute#23, order_second#24, is_weekend#25, order_status#26, return_reason#27, customer_id#28, customer_name#29, gender#30, age#31, customer_segment#32, country#33, city#34, customer_loyalty_score#35, total_orders_by_customer#36, account_creation_date#37, product_id#38, product_name#39, category#40, ... 38 more fields]
```

```
+-- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```



```

== Optimized Logical Plan ==
Window [sum(total_price_usd#50) windowSpecdefinition(order_date#18 ASC NULLS FIRST, specifiedwindowframe(RowFrame,
unboundedpreceding$(), currentrow$())) AS cumulative_revenue#1473], [order_date#18 ASC NULLS FIRST]
+- Project [order_date#18, total_price_usd#50]
   +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv

== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- Window [sum(total_price_usd#50) windowSpecdefinition(order_date#18 ASC NULLS FIRST, specifiedwindowframe(RowFrame,
unboundedpreceding$(), currentrow$())) AS cumulative_revenue#1473], [order_date#18 ASC NULLS FIRST]
   +- Sort [order_date#18 ASC NULLS FIRST], false, 0
      +- Exchange SinglePartition, ENSURE_REQUIREMENTS, [plan_id=675]
         +- FileScan csv [order_date#18,total_price_usd#50] Batched: false, DataFilters: [], Format: CSV, Location:
InMemoryFileIndex(1 paths)[file:/home/yousef/spark_project/data/ecommerce.csv], PartitionFilters: [], PushedFilters:
 [], ReadSchema: struct<order_date:timestamp,total_price_usd:double>

```

QUERY 10

```

110
111 # --- Q10: Subquery ---
112 t = time.time()
113 customer_spend = df.groupBy("customer_id") \
114                     .agg(F.sum("total_price_usd").alias("total_spend"))
115
116 avg_spend = customer_spend.agg(F.avg("total_spend")).collect()[0][0]
117
118 q10 = customer_spend.filter(F.col("total_spend") > avg_spend) \
119                     .orderBy(F.desc("total_spend"))
120
121 print(f"\nQ10 Average Spend: {avg_spend:.2f}")
122 run_query("Q10 - High-value customers", q10)
123 timings['Q10_time'] = time.time() - t
124

```

Q10 Average Spend: 406.56

=====

Q10 - High-value customers RESULT

=====

customer_id	total_spend
CUS-CSPL3	4267.91
CUS-WXS3A	4043.9300000000003
CUS-DK609	3888.2
CUS-A67K3	3735.48
CUS-P1D03	3542.4300000000003
CUS-5XVL1	3506.8
CUS-96P9R	3495.37
CUS-Q9X3X	3467.6400000000003
CUS-A20PJ	3422.48
CUS-V010N	3334.0200000000004

only showing top 10 rows

Q10 - High-value customers EXPLAIN PLAN

== Parsed Logical Plan ==

'Sort ['total_spend DESC NULLS LAST], true

+ Filter (total_spend#1619 > 406.5607034664071)

+ Aggregate [customer_id#28], [customer_id#28, sum(total_price_usd#50) AS total_spend#1619]

+ Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv

== Analyzed Logical Plan ==

customer_id: string, total_spend: double

Sort [total_spend#1619 DESC NULLS LAST], true

+ Filter (total_spend#1619 > 406.5607034664071)

+ Aggregate [customer_id#28], [customer_id#28, sum(total_price_usd#50) AS total_spend#1619]

+ Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv


```

== Optimized Logical Plan ==
Sort [total_spend#1619 DESC NULLS LAST], true
+- Filter (isnotnull(total_spend#1619) AND (total_spend#1619 > 406.5607034664071))
  +- Aggregate [customer_id#28], [customer_id#28, sum(total_price_usd#50) AS total_spend#1619]
    +- Project [customer_id#28, total_price_usd#50]
      +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv

== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- Sort [total_spend#1619 DESC NULLS LAST], true, 0
  +- Exchange rangepartitioning(total_spend#1619 DESC NULLS LAST, 8), ENSURE_REQUIREMENTS, [plan_id=842]
    +- Filter (isnotnull(total_spend#1619) AND (total_spend#1619 > 406.5607034664071))
      +- HashAggregate(keys=[customer_id#28], functions=[sum(total_price_usd#50)], output=[customer_id#28, total_spend#1619])
        +- Exchange hashpartitioning(customer_id#28, 8), ENSURE_REQUIREMENTS, [plan_id=838]
          +- HashAggregate(keys=[customer_id#28], functions=[partial_sum(total_price_usd#50)], output=[customer_id#28, sum#1632])
            +- FileScan csv [customer_id#28,total_price_usd#50] Batched: false, DataFilters: [], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/yousef/spark_project/data/ecommerce.csv], PartitionFilters: [], PushedFilters: [], ReadSchema: struct<customer_id:string,total_price_usd:double>

```

SQL

SHUFFLE SIZE AND MEMORY USAGE

▼ Completed Stages (17)

Page: 11 Pages. Jump to 1. Show 100 items in a page. Go

Stage Id ▼	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
23	showString at NativeMethodAccessorImpl.java:0 +details	2026/04/24 19:37:18	84 ms	1/1			3.3 KiB	
21	showString at NativeMethodAccessorImpl.java:0 +details	2026/04/24 19:37:08	10 s	4/4	391.6 MiB			3.3 KiB
20	showString at NativeMethodAccessorImpl.java:0 +details	2026/04/24 19:37:05	2 s	1/1			14.6 MiB	
18	showString at NativeMethodAccessorImpl.java:0 +details	2026/04/24 19:36:50	15 s	4/4	391.6 MiB			14.6 MiB
17	showString at NativeMethodAccessorImpl.java:0 +details	2026/04/24 19:36:49	0.1 s	1/1			1590.0 B	
15	showString at NativeMethodAccessorImpl.java:0 +details	2026/04/24 19:36:38	11 s	4/4	391.6 MiB			1590.0 B
14	showString at NativeMethodAccessorImpl.java:0 +details	2026/04/24 19:36:37	0.1 s	1/1			2.3 KiB	
12	showString at NativeMethodAccessorImpl.java:0 +details	2026/04/24 19:36:28	9 s	4/4	391.6 MiB			2.3 KiB
11	showString at NativeMethodAccessorImpl.java:0 +details	2026/04/24 19:36:27	0.2 s	1/1			1144.0 B	
9	showString at NativeMethodAccessorImpl.java:0 +details	2026/04/24 19:36:18	9 s	4/4	391.6 MiB			1144.0 B
8	showString at NativeMethodAccessorImpl.java:0 +details	2026/04/24 19:36:16	75 ms	1/1			236.0 B	
6	showString at NativeMethodAccessorImpl.java:0 +details	2026/04/24 19:36:07	9 s	4/4	391.6 MiB			236.0 B
5	showString at NativeMethodAccessorImpl.java:0 +details	2026/04/24 19:36:05	0.9 s	1/1	64.0 KiB			
4	count at NativeMethodAccessorImpl.java:0 +details	2026/04/24 19:36:03	0.3 s	1/1			236.0 B	
2	count at NativeMethodAccessorImpl.java:0 +details	2026/04/24 19:35:59	3 s	4/4	391.6 MiB			236.0 B
1	csv at NativeMethodAccessorImpl.java:0 +details	2026/04/24 19:35:35	23 s	4/4	391.6 MiB			
0	csv at NativeMethodAccessorImpl.java:0 +details	2026/04/24 19:35:32	0.8 s	1/1	64.0 KiB			

Trash

Page: 11 Pages. Jump to 1. Show 100 items in a page. Go

36	showString at NativeMethodAccessorImpl.java:0	+details	Unknown	Unknown	0/4				
----	---	--------------------------	---------	---------	-----	--	--	--	--

Page: 1 Pages. Jump to . Show items in a page. [Go](#)

▼ Completed Stages (24)

Page: 1 Pages. Jump to . Show items in a page. [Go](#)

Stage Id ▼	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
35	showString at NativeMethodAccessorImpl.java:0	+details	2026/04/24 19:38:20	2 s	1/1		14.6 MiB	19.3 MiB
33	showString at NativeMethodAccessorImpl.java:0	+details	2026/04/24 19:38:05	15 s	4/4	391.6 MiB		14.6 MiB
32	showString at NativeMethodAccessorImpl.java:0	+details	2026/04/24 19:38:04	0.1 s	1/1		274.0 B	
29	showString at NativeMethodAccessorImpl.java:0	+details	2026/04/24 19:37:57	7 s	1/1		9.4 MiB	274.0 B
27	showString at NativeMethodAccessorImpl.java:0	+details	2026/04/24 19:37:36	20 s	4/4	391.6 MiB		9.4 MiB
Help	showString at NativeMethodAccessorImpl.java:0	+details	2026/04/24 19:37:32	0.3 s	1/1		8.7 KiB	
24	showString at NativeMethodAccessorImpl.java:0	+details	2026/04/24 19:37:19	12 s	4/4	391.6 MiB		8.7 KiB
23	showString at NativeMethodAccessorImpl.java:0	+details	2026/04/24 19:37:18	84 ms	1/1		3.3 KiB	
21	showString at NativeMethodAccessorImpl.java:0	+details	2026/04/24 19:37:08	10 s	4/4	391.6 MiB		3.3 KiB
20	showString at NativeMethodAccessorImpl.java:0	+details	2026/04/24 19:37:05	2 s	1/1		14.6 MiB	
18	showString at NativeMethodAccessorImpl.java:0	+details	2026/04/24 19:36:50	15 s	4/4	391.6 MiB		14.6 MiB
17	showString at NativeMethodAccessorImpl.java:0	+details	2026/04/24 19:36:49	0.1 s	1/1		1590.0 B	
15	showString at NativeMethodAccessorImpl.java:0	+details	2026/04/24 19:36:38	11 s	4/4	391.6 MiB		1590.0 B
14	showString at NativeMethodAccessorImpl.java:0	+details	2026/04/24 19:36:37	0.1 s	1/1		2.3 KiB	
12	showString at NativeMethodAccessorImpl.java:0	+details	2026/04/24 19:36:28	9 s	4/4	391.6 MiB		2.3 KiB
11	showString at NativeMethodAccessorImpl.java:0	+details	2026/04/24 19:36:27	0.2 s	1/1		1144.0 B	

QUERY 1

```

4
5 "Q1 - Filter Completed > 500": ""
6   SELECT COUNT(*) as completed_over_500
7   FROM ecommerce
8   WHERE order_status = 'Completed' AND total_price_usd > 500
9

```



```

=====
Running Q1 via Spark SQL...
+-----+
|completed_over_500|
+-----+
|          198151|
+-----+

Q1 Time: 8.803s
=== Q1 EXPLAIN ===
== Parsed Logical Plan ==
'Project ['COUNT(1) AS completed_over_500#520]
+- 'Filter (('order_status = Completed) AND ('total_price_usd > 500))
   +- 'UnresolvedRelation [ecommerce], [], false

== Analyzed Logical Plan ==
completed_over_500: bigint
Aggregate [count(1) AS completed_over_500#520L]
+- Filter ((order_status#26 = Completed) AND (total_price_usd#50 > cast(500 as double)))
   +- SubqueryAlias ecommerce
      +- View ('ecommerce', [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,count_ry#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,sub_category#41,brand#42,product_rating_avg#43,product_reviews_count#44,stock_quantity#45,unit_price_usd#46,quantity#47,discount_percent#48,discount_amount_usd#49,total_price_usd#50,cost_usd#51,profit_usd#52,tax_usd#53,currency#54,payment_method#55,payment_status#56,installment_plan#57,shipping_method#58,shipping_cost_usd#59,delivery_days#60,shipping_country#61,warehouse_location#62,delivery_status#63,rating#64,review_sentiment#65,customer_feedback#66,coupon_used#67,coupon_code#68,campaign_source#69,device_type#70,traffic_source#71,session_duration_minutes#72,pages_vlsited#73,abandoned_cart_before#74,fraud_risk_score#75,profit_margin_percent#76,order_priority#77,support_ticket_created#78])
      +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
                                          Go to Settings to activate Windows.

```

```

== Optimized Logical Plan ==
Aggregate [count(1) AS completed_over_500#520L]
+- Project
   +- Filter (((isnotnull(order_status#26) AND isnotnull(total_price_usd#50)) AND ((order_status#26 = Completed) AND (total_price_usd#50 > 500.0))))
      +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 fields] csv

== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- HashAggregate(keys=[], functions=[count(1)], output=[completed_over_500#520L])
   +- Exchange SinglePartition, ENSURE_REQUIREMENTS, [plan_id=123]
      +- HashAggregate(keys=[], functions=[partial_count(1)], output=[count#527L])
         +- Project
            +- Filter (((isnotnull(order_status#26) AND isnotnull(total_price_usd#50)) AND (order_status#26 = Completed)) AND (total_price_usd#50 > 500.0))
               +- FileScan csv [order_status#26,total_price_usd#50] Batched: false, DataFilters: [isnotnull(order_status#26), isnotnull(total_price_usd#50), (order_status#26 = Completed), (total..., Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/yousef/spark_project/data/ecommerce.csv], PartitionFilters: [], PushedFilters: [IsNotNull(order_status), IsNotNull(total_price_usd), EqualTo(order_status,Completed), GreaterTha..., ReadSchema: struct<order_status:string,total_price_usd:double>

```

QUERY 2


```

40
41 "Q2 - Revenue by category": ""
42     SELECT category, ROUND(SUM(total_price_usd),2) AS total_revenue
43     FROM ecommerce
44     GROUP BY category
45     ORDER BY total_revenue DESC
46     ""

```

=====

Q2 - Revenue by category RESULT

=====

category	total_revenue
Electronics	1.3986598694E8
Hone	8.782867776E7
Sports	7.291747085E7
Health	5.746792555E7
Clothing	4.52057959E7

Q2 - Revenue by category EXPLAIN PLAN

== Parsed Logical Plan ==

'Sort ['total_revenue DESC NULLS LAST], true

+-- 'Aggregate ['category], ['category, 'ROUND('SUM('total_price_usd), 2) AS total_revenue#532]

+-- 'UnresolvedRelation [ecommerce], [], false

== Analyzed Logical Plan ==

category: string, total_revenue: double

Sort [total_revenue#532 DESC NULLS LAST], true

+-- Aggregate [category#40], [category#40, round(sum(total_price_usd#80), 2) AS total_revenue#532]

+-- SubqueryAlias ecommerce

```

== Optimized Logical Plan ==
Sort [total_revenue#532 DESC NULLS LAST], true
+- Aggregate [category#40], [category#40, round(sum(total_price_usd#50), 2) AS total_revenue#532]
   +- Project [category#40, total_price_usd#50]
      +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,
is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city
#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38
more fields] csv

== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- Sort [total_revenue#532 DESC NULLS LAST], true, 0
   +- Exchange rangepartitioning(total_revenue#532 DESC NULLS LAST, 8), ENSURE_REQUIREMENTS, [plan_id=188]
      +- HashAggregate(keys=[category#40], functions=[sum(total_price_usd#50)], output=[category#40, total_revenue#532])
         +- Exchange hashpartitioning(category#40, 8), ENSURE_REQUIREMENTS, [plan_id=185]
            +- HashAggregate(keys=[category#40], functions=[partial_sum(total_price_usd#50)], output=[category#40, sum#543])
               +- FileScan csv [category#40,total_price_usd#50] Batched: false, DataFilters: [], Format: CSV, Location: InMemoryFile
Index(1 paths)[file:/home/yousef/spark_project/data/ecommerce.csv], PartitionFilters: [], PushedFilters: [], ReadSchema: struct<cate
gory:string,total_price_usd:double>

```

QEURY 3

```

Terminal 33 - Avg order value by country": ""
49      SELECT country, ROUND(AVG(total_price_usd),2) AS avg_order_value
50      FROM ecommerce
51      GROUP BY country
52      ORDER BY avg_order_value DESC
53

```

Running Q3 via Spark SQL...

```
+-----+-----+
| country|avg_order_value|
+-----+-----+
| Italy|405.73|
| Australia|404.23|
| Spain|403.61|
| United States|403.41|
| Netherlands|403.24|
| United Kingdom|403.15|
| Belgium|402.73|
| Germany|402.64|
| Canada|401.94|
| France|401.71|
+-----+-----+
```

Q3 Time: 8.350s

=== Q3 EXPLAIN ===

== Parsed Logical Plan ==

```
Sort [avg_order_value DESC NULLS LAST], true
+- Aggregate [country#33], [country#33, round(avg(total_price_usd), 2) AS avg_order_value#551]
   +- 'UnresolvedRelation [ecommerce], [], false
```

== Analyzed Logical Plan ==

```
country: string, avg_order_value: double
Sort [avg_order_value#551 DESC NULLS LAST], true
+- Aggregate [country#33], [country#33, round(avg(total_price_usd#50), 2) AS avg_order_value#551]
   +- SubqueryAlias ecommerce
      +- View ('ecommerce', [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,sub_category#41,brand#42,product_rating_avg#43,product_reviews_count#44,stock_quantity#45,unit_price_usd#46,quantity#47,discount_percent#48,discount_amount_usd#49,total_price_usd#50,cost_usd#51,profit_usd#52,tax_usd#53,currency#54,payment_method#55,payment_status#56,installment_plan#57,shipping_method#58,shipping_cost_usd#59,delivery_days#60,shipping_country#61,warehouse_location#62,delivery_status#63,rating#64,review_sentiment#65,customer_feedback#66,coupon_used#67,coupon_code#68,campaign_source#69,device_type#70,traffic_source#71,session_duration_minutes#72,pages_visited#73,abandoned_cart_before#74,fraud_risk_score#75,profit_margin_percent#76,order_priority#77,support_ticket_created#78])
      +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

== Optimized Logical Plan ==

```
Sort [avg_order_value#551 DESC NULLS LAST], true
+- Aggregate [country#33], [country#33, round(avg(total_price_usd#50), 2) AS avg_order_value#551]
   +- Project [country#33, total_price_usd#50]
      +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

== Physical Plan ==

```
AdaptiveSparkPlan isFinalPlan=false
+- Sort [avg_order_value#551 DESC NULLS LAST], true, 0
   +- Exchange rangepartitioning(avg_order_value#551 DESC NULLS LAST, 8), ENSURE_REQUIREMENTS, [plan_id=253]
      +- HashAggregate(keys=[country#33], Functions=[avg(total_price_usd#50)], output=[country#33, avg_order_value#551])
         +- Exchange hashpartitioning(country#33, 8), ENSURE_REQUIREMENTS, [plan_id=250]
            +- HashAggregate(keys=[country#33], Functions=[partial_avg(total_price_usd#50)], output=[country#33, sum#563, count#564])
               +- FileScan csv [country#33,total_price_usd#50] Batched: false, DataFilters: [], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/yousef/spark_project/data/ecommerce.csv], PartitionFilters: [], PushedFilters: [], ReadSchema: struct<count
```

QUERY 4

```
54
55 "Q4 - Orders by payment method": ""
56     SELECT payment_method, COUNT(*) AS order_count
57     FROM ecommerce
58     GROUP BY payment_method
59     ORDER BY order_count DESC
60 ""
```



```
Running Q4 via Spark SQL...
+-----+
|payment_method|order_count|
+-----+
|Debit Card|200805|
|Bank Transfer|200213|
|PayPal|199881|
|Apple Pay|199814|
|Credit Card|199410|
+-----+

Q4 Time: 8.159s
=== Q4 EXPLAIN ===
== Parsed Logical Plan ==
'Sort ['order_count DESC NULLS LAST], true
+- 'Aggregate [payment_method], [payment_method, 'COUNT(1) AS order_count#574]
   +- 'UnresolvedRelation [ecommerce], [], false

== Analyzed Logical Plan ==
payment_method: string, order_count: bigint
Sort [order_count#574L DESC NULLS LAST], true
+- Aggregate [payment_method#55], [payment_method#55, count(1) AS order_count#574L]
   +- SubqueryAlias ecommerce
      +- View ('ecommerce', [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,sub_category#41,brand#42,product_rating_avg#43,product_reviews_count#44,stock_quantity#45,unit_price_usd#46,quantity#47,discount_percent#48,discount_amount_usd#49,total_price_usd#50,cost_usd#51,profit_usd#52,tax_usd#53,currency#54,payment_method#55,payment_status#56,installment_plan#57,shipping_method#58,shipping_cost_usd#59,delivery_days#60,shipping_country#61,warehouse_location#62,delivery_status#63,rating#64,review_sentiment#65,customer_feedback#66,coupon_used#67,coupon_code#68,campaign_source#69,device_type#70,traffic_source#71,subscription_start_date#72,subscription_end_date#73,subscription_status#74,fraud_risk_score#75,profit_margin_percent#76,order_id#77,support_ticket_created#78])
      +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv

Activate Windows
```

```
== Optimized Logical Plan ==
Sort [order_count#574L DESC NULLS LAST], true
+- Aggregate [payment_method#55], [payment_method#55, count(1) AS order_count#574L]
   +- Project [payment_method#55]
      +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv

== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- Sort [order_count#574L DESC NULLS LAST], true, 0
   +- Exchange rangepartitioning(order_count#574L DESC NULLS LAST, 8), ENSURE_REQUIREMENTS, [plan_id=318]
      +- HashAggregate(keys=[payment_method#55], functions=[count(1)], output=[payment_method#55, order_count#574L])
         +- Exchange hashpartitioning(payment_method#55, 8), ENSURE_REQUIREMENTS, [plan_id=315]
            +- HashAggregate(keys=[payment_method#55], functions=[partial_count(1)], output=[payment_method#55, count#585L])
               +- FileScan csv [payment_method#55] Batched: false, DataFilters: [], Format: CSV, Location: InMemoryFileIndex(1 paths) [file:/home/yousef/spark_project/data/ecommerce.csv], PartitionFilters: [], PushedFilters: [], ReadSchema: struct<payment_method:string>
```

QUERY 5

```
61
62 "Q5 - Max order per customer": ""
63 SELECT customer_id, MAX(total_price_usd) AS max_order
64 FROM ecommerce
65 GROUP BY customer_id
66 ORDER BY max_order DESC
67 LIMIT 10
68 ""
```

Running Q5 via Spark SQL...

```
+-----+
|customer_id|max_order|
+-----+
|CUS-A9QVX| 2499.75|
|CUS-SF2Q8| 2499.45|
|CUS-WKXXV| 2499.2|
|CUS-LY7W1| 2499.15|
|CUS-ESEP9| 2498.95|
|CUS-8TUG3| 2498.8|
|CUS-F3THO| 2498.75|
|CUS-UR59J| 2498.45|
|CUS-9S8VR| 2497.8|
|CUS-1BE0M| 2497.6|
+-----+
```

Q5 Time: 11.008s

=== Q5 EXPLAIN ===

== Parsed Logical Plan ==

GlobalLimit 10

+ LocalLimit 10

```
+-- Sort ['max_order DESC NULLS LAST], true
+-- Aggregate ['customer_id'], ['customer_id', 'MAX('total_price_usd') AS max_order#590]
+-- UnresolvedRelation [ecommerce], [], false
```

== Analyzed Logical Plan ==

customer_id: string, max_order: double

GlobalLimit 10

+ LocalLimit 10

```
+-- Sort [max_order#590 DESC NULLS LAST], true
+-- Aggregate [customer_id#28], [customer_id#28, max(total_price_usd#50) AS max_order#590]
+-- SubqueryAlias ecommerce
```

```
+-- View ('ecommerce', [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,sub_category#41,brand#42,product_rating_avg#43,product_reviews_count#44,stock_quantity#45,unit_price_usd#46,quantity#47,dls_count_percent#48,discount_amount_usd#49,total_price_usd#50,cost_usd#51,profit_usd#52,tax_usd#53,currency#54,payment_method#55,payment_status#56,installment_plan#57,shipping_method#58,shipping_cost_usd#59,delivery_days#60,shipping_country#61,warehouse_location#62,delivery_status#63,rating#64,review_sentiment#65,customer_feedback#66,coupon_used#67,coupon_code#68,campaign_source#69,device_type#70,traffic_source#71,session_duration_minutes#72,pages_visited#73,abandoned_cart_before#74,fraud_risk_score#75,profit_margin_percent#76,order_priority#77,support_ticket_created#78])
```

```
+-- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

== Optimized Logical Plan ==

GlobalLimit 10

+ LocalLimit 10

```
+-- Sort [max_order#590 DESC NULLS LAST], true
+-- Aggregate [customer_id#28], [customer_id#28, max(total_price_usd#50) AS max_order#590]
+-- Project [customer_id#28, total_price_usd#50]
```

```
+-- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

== Physical Plan ==

AdaptiveSparkPlan isFinalPlan=false

+ TakeOrderedAndProject(limit=10, orderBy=[max_order#590 DESC NULLS LAST], output=[customer_id#28,max_order#590])

```
+-- HashAggregate(keys=[customer_id#28], functions=[max(total_price_usd#50)], output=[customer_id#28, max_order#590])
```

```
+-- Exchange hashpartitioning(customer_id#28, 8), ENSURE_REQUIREMENTS, [plan_id=1379] Create Windows
```

```
+-- HashAggregate(keys=[customer_id#28], functions=[partial_max(total_price_usd#50)], output=[customer_id#28,max_order#590])
```

```
+-- FileScan csv [customer_id#28,total_price_usd#50] Batched: false, DataFilters: [], Format: CSV, Location: InMemoryFile
Index(1 paths)[file:/home/yousef/spark_project/data/ecommerce.csv], PartitionFilters: [], PushedFilters: [], ReadSchema: struct<cust
```

QUERY 6

```
Help
70  "Q6 - Orders per month": ""
71      SELECT DATE_FORMAT(order_date, 'yyyy-MM') AS month,
72             COUNT(*) AS order_count
73      FROM ecommerce
74      GROUP BY month
75      ORDER BY month
76  ""
```


=====

Running Q6 via Spark SQL...

```
+-----+-----+
| month|order_count|
+-----+-----+
|2024-02|      37052|
|2024-03|      42716|
|2024-04|      41415|
|2024-05|      40756|
|2024-06|      42255|
|2024-07|      42241|
|2024-08|      41035|
|2024-09|      42245|
|2024-10|      40800|
+-----+-----+
```

only showing top 10 rows

Q6 Time: 9.298s

=== Q6 EXPLAIN ===

== Parsed Logical Plan ==

```
'Sort ['month ASC NULLS FIRST], true
+- 'Aggregate ['month], ['DATE_FORMAT('order_date, yyyy-MM) AS month#609, 'COUNT(1) AS order_count#610]
   +- 'UnresolvedRelation [ecommerce], [], false
```

== Analyzed Logical Plan ==

```
month: string, order_count: bigint
Sort [month#609 ASC NULLS FIRST], true
+- Aggregate [date_format(order_date#18, yyyy-MM, Some(Africa/Cairo)), [date_format(order_date#18, yyyy-MM, Some(Africa/Cairo)) AS month#609, count(1) AS order_count#610]
   +- SubqueryAlias ecommerce
      +- View ('ecommerce', [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,sub_category#41,brand#42,product_rating_avg#43,product_reviews_count#44,stock_quantity#45,unit_price_usd#46,quantity#47,discount_percent#48,discount_amount_usd#49,total_price_usd#50,cost_usd#51,profit_usd#52,tax_usd#53,currency#54,payment_method#55,payment_status#56,installment_plan#57,shipping_method#58,shipping_cost_usd#59,delivery_days#60,shipping_country#61,warehouse_location#62,delivery_status#63,rating#64,review_sentiment#65,customer_feedback#66,coupon_used#67,coupon_code#68,campaign_source#69,device_type#70,traffic_source#71,session_duration_minutes#72,pages_visited#73,abandoned_cart_before#74,fraud_risk_score#75,profit_margin_percent#76,order_priority#77,support_ticket_created#78])
      +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

== Optimized Logical Plan ==

```
Sort [month#609 ASC NULLS FIRST], true
+- Aggregate [_groupingexpression#627], [_groupingexpression#627 AS month#609, count(1) AS order_count#610]
   +- Project [date_format(order_date#18, yyyy-MM, Some(Africa/Cairo)) AS _groupingexpression#627]
      +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
```

== Physical Plan ==

```
AdaptiveSparkPlan isFinalPlan=false
+- Sort [month#609 ASC NULLS FIRST], true, 0
   +- Exchange rangepartitioning(month#609 ASC NULLS FIRST, 8), ENSURE_REQUIREMENTS, [plan_id=454]
      +- HashAggregate(keys=[_groupingexpression#627], functions=[count(1)], output=[month#609, order_count#610])
         +- Exchange hashpartitioning(_groupingexpression#627, 8), ENSURE_REQUIREMENTS, [plan_id=451]
            +- HashAggregate(keys=[_groupingexpression#627], functions=[partial count(1)], output=[_groupingexpression#627, count#62])
               +- Project [date_format(order_date#18, yyyy-MM, Some(Africa/Cairo)) AS _groupingexpression#627]
                  +- FileScan csv [order_date#18] Batched: false, DataFilters: [], Format: CSV, Location: InMemoryFileIndex(1 paths) [file:/home/yousef/spark_project/data/ecommerce.csv], PartitionFilters: [], PushedFilters: [], ReadSchema: struct<order_date:timestamp>
```


QUERY 7

```
77
78 "Q7 - Revenue by category & country": ""
79     SELECT category, country,
80           ROUND(SUM(total_price_usd),2) AS revenue
81     FROM ecommerce
82     GROUP BY category, country
83     ORDER BY revenue DESC
84     LIMIT 10
85     ""
```

```
=====
Running Q7 via Spark SQL...
+-----+-----+
| category|country|revenue|
+-----+-----+
|Electronics|Netherlands|1.416504796E7|
|Electronics|Italy|1.410976446E7|
|Electronics|Belgium|1.406366308E7|
|Electronics|Spain|1.400882483E7|
|Electronics|United Kingdom|1.400408099E7|
|Electronics|Canada|1.397057633E7|
|Electronics|United States|1.396531785E7|
|Electronics|Australia|1.395247316E7|
|Electronics|Germany|1.387607852E7|
|Electronics|France|1.375015976E7|
+-----+-----+

Q7 Time: 8.018s
=== Q7 EXPLAIN ===
== Parsed Logical Plan ==
'GlobalLimit 10
+- 'LocalLimit 10
   +- 'Sort ['revenue DESC NULLS LAST], true
      +- 'Aggregate ['category, 'country'], ['category, 'country, 'ROUND('SUM('total_price_usd), 2) AS revenue#628]
         +- 'UnresolvedRelation [ecommerce], [], false
```

```

== Analyzed Logical Plan ==
category: string, country: string, revenue: double
GlobalLimit 10
+- LocalLimit 10
  +- Sort [revenue#628 DESC NULLS LAST], true
    +- Aggregate [category#40, country#33], [category#40, country#33, round(sum(total_price_usd#50), 2) AS revenue#628]
      +- SubqueryAlias ecommerce
        +- View ('ecommerce', [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,sub_category#41,brand#42,product_rating_avg#43,product_reviews_count#44,stock_quantity#45,unit_price_usd#46,quantity#47,discount_percent#48,discount_amount_usd#49,total_price_usd#50,cost_usd#51,profit_usd#52,tax_usd#53,currency#54,payment_method#55,payment_status#56,installment_plan#57,shipping_method#58,shipping_cost_usd#59,delivery_days#60,shipping_country#61,warehouse_location#62,delivery_status#63,rating#64,review_sentiment#65,customer_feedback#66,coupon_used#67,coupon_code#68,campaign_source#69,device_type#70,traffic_source#71,session_duration_minutes#72,pages_visited#73,abandoned_cart_before#74,fraud_risk_score#75,profit_margin_percent#76,order_priority#77,support_ticket_created#78])
        +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv

== Optimized Logical Plan ==
GlobalLimit 10
+- LocalLimit 10
  +- Sort [revenue#628 DESC NULLS LAST], true
    +- Aggregate [category#40, country#33], [category#40, country#33, round(sum(total_price_usd#50), 2) AS revenue#628]
      +- Project [country#33, category#40, total_price_usd#50]
        +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv

== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- TakeOrderedAndProject(limit=10, orderBy=[revenue#628 DESC NULLS LAST], output=[category#40,country#33,revenue#628])
  +- HashAggregate(keys=[category#40, country#33], functions=[sum(total_price_usd#50)], output=[category#40, country#33, revenue#628])
    +- Exchange hashpartitioning(category#40, country#33, 8), ENSURE_REQUIREMENTS, [plan_id=515]
      +- HashAggregate(keys=[category#40, country#33], functions=[partial_sum(total_price_usd#50)], output=[category#40, country#

```

QUERY 8

```
"Q8 - Distinct products per category": ""  
SELECT category, COUNT(DISTINCT product_id) AS distinct_products  
FROM ecommerce  
GROUP BY category  
ORDER BY distinct_products DESC
```

```
=====  
Running Q8 via Spark SQL...  
+-----+-----+  
| category|distinct_products|  
+-----+-----+  
| Sports| 189088|  
| Home| 189059|  
| Electronics| 188408|  
| Clothing| 188183|  
| Health| 188127|  
+-----+-----+
```

```
=====  
Running Q8 via Spark SQL...  
+-----+-----+  
| category|distinct_products|  
+-----+-----+  
| Sports| 189088|  
| Home| 189059|  
| Electronics| 188408|  
| Clothing| 188183|  
| Health| 188127|  
+-----+-----+  
  
Q8 Time: 11.101s  
=== Q8 EXPLAIN ===  
== Parsed Logical Plan ==  
'Sort ['distinct_products DESC NULLS LAST], true  
+- 'Aggregate ['category], ['category, 'COUNT(distinct 'product_id) AS distinct_products#653]  
+- 'UnresolvedRelation [ecommerce], [], false  
  
== Analyzed Logical Plan ==  
category: string, distinct_products: bigint  
Sort [distinct_products#653L DESC NULLS LAST], true  
+- Aggregate [category#40], [category#40, count(distinct product_id#38) AS distinct_products#653L]  
+- SubqueryAlias ecommerce  
+- View ('ecommerce', [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order  
_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,count  
ry#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#  
40,sub_category#41,brand#42,product_rating_avg#43,product_reviews_count#44,stock_quantity#45,unit_price_usd#46,quantity#47,discount  
percent#48,discount_amount_usd#49,total_price_usd#50,cost_usd#51,profit_usd#52,tax_usd#53,currency#54,payment_method#55,payment_stat  
us#56,installment_plan#57,shipping_method#58,shipping_cost_usd#59,delivery_days#60,shipping_country#61,warehouse_location#62,deliver  
y_status#63,rating#64,review_sentiment#65,customer_feedback#66,coupon_used#67,coupon_code#68,campaign_source#69,device_type#70,traff  
ic_source#71,session_duration_minutes#72,pages_visited#73,abandoned_cart_before#74,fraud_risk_score#75,profit_margin_percent#76,orde  
r_priority#77,support_ticket_created#78])  
+- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second  
#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,c  
ity#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,...  
38 more fields] csv
```



```

== Optimized Logical Plan ==
Sort [distinct_products#653L DESC NULLS LAST], true
+- Aggregate [category#40], [category#40, count(distinct product_id#38) AS distinct_products#653L]
  +- Project [product_id#38, category#40]
    +- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,
is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city
#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38
more fields] csv

== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- Sort [distinct_products#653L DESC NULLS LAST], true, 0
  +- Exchange rangepartitioning(distinct_products#653L DESC NULLS LAST, 8), ENSURE_REQUIREMENTS, [plan_id=647]
    +- HashAggregate(keys=[category#40], functions=[count(distinct product_id#38)], output=[category#40, distinct_products#653L])
      +- Exchange hashpartitioning(category#40, 8), ENSURE_REQUIREMENTS, [plan_id=644]
        +- HashAggregate(keys=[category#40], functions=[partial_count(distinct product_id#38)], output=[category#40, count#665L])
          +- HashAggregate(keys=[category#40, product_id#38], functions=[], output=[category#40, product_id#38])
            +- Exchange hashpartitioning(category#40, product_id#38, 8), ENSURE_REQUIREMENTS, [plan_id=640]
              +- HashAggregate(keys=[category#40, product_id#38], functions=[], output=[category#40, product_id#38])
                +- FileScan csv [product_id#38,category#40] Batched: false, DataFilters: [], Format: CSV, Location: InMemory
FileIndex(1 paths)[file:/home/yousef/spark_project/data/ecommerce.csv], PartitionFilters: [], PushedFilters: [], ReadSchema: struct<
product_id:string,category:string>

```

QUERY 9

```

93
94 "Q9 - Window function ranking": ""
95     SELECT customer_id,
96            SUM(total_price_usd) AS total_spend,
97            RANK() OVER (ORDER BY SUM(total_price_usd) DESC) AS spend_rank
98     FROM ecommerce
99     GROUP BY customer_id
100    ORDER BY spend_rank
101    LIMIT 20
102

```


Running Q9 via Spark SQL...

```
+-----+
|customer_id|      total_spend|spend_rank|
+-----+
|CUS-CSPL3|      4267.91|          1|
|CUS-WX53A|4043.9300000000003|          2|
|CUS-DK609|      3888.2|          3|
|CUS-A67K3|      3735.48|          4|
|CUS-P1D03|3542.4300000000003|          5|
|CUS-SXXVL1|      3506.8|          6|
|CUS-96P9R|      3495.37|          7|
|CUS-Q9X3X|3467.6400000000003|          8|
|CUS-A20PJ|      3422.48|          9|
|CUS-V010N|3334.0200000000004|         10|
+-----+
```

only showing top 10 rows

Q9 Time: 13.157s

=== Q9 EXPLAIN ===

== Parsed Logical Plan ==

'GlobalLimit 20

+-- 'LocalLimit 20

+- 'Sort ['spend_rank ASC NULLS FIRST], true

+- 'Aggregate ['customer_id], ['customer_id, 'SUM('total_price_usd) AS total_spend#677, 'RANK() windowSpecDefinition('SUM('total_price_usd) DESC NULLS LAST, unspecifiedFrame()) AS spend_rank#678]

+- 'UnresolvedRelation [ecommerce], [], false

== Analyzed Logical Plan ==

customer_id: string, total_spend: double, spend_rank: int

GlobalLimit 20

+-- LocalLimit 20

+- Sort [spend_rank#678 ASC NULLS FIRST], true

+- Project [customer_id#28, total_spend#677, spend_rank#678]

+- Window [rank(_w0#683) windowSpecDefinition(_w0#683 DESC NULLS LAST, specifiedWindowFrame(RowFrame, unboundedPrecedingS(), currentRowS())) AS spend_rank#678], [_w0#683 DESC NULLS LAST]

+- Aggregate [customer_id#28], [customer_id#28, sum(total_price_usd#50) AS total_spend#677, sum(total_price_usd#50) AS _w0#683]

+- Aggregate [customer_id#28], [customer_id#28, sum(total_price_usd#50) AS total_spend#677, sum(total_price_usd#50) AS _w0#683]

+- SubqueryAlias ecommerce

+- View ('ecommerce', [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,sub_category#41,brand#42,product_rating_avg#43,product_reviews_count#44,stock_quantity#45,unit_price_usd#46,quantity#47,discount_percent#48,discount_amount_usd#49,total_price_usd#50,cost_usd#51,profit_usd#52,tax_usd#53,currency#54,payment_method#55,payment_status#56,installment_plan#57,shipping_method#58,shipping_cost_usd#59,delivery_days#60,shipping_country#61,warehouse_location#62,delivery_status#63,rating#64,review_sentiment#65,customer_feedback#66,coupon_used#67,coupon_code#68,campaign_source#69,device_type#70,traffic_source#71,session_duration_minutes#72,pages_visited#73,abandoned_cart_before#74,fraud_risk_score#75,profit_margin_percent#76,order_priority#77,support_ticket_created#78])

+- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv

== Optimized Logical Plan ==

GlobalLimit 20

+-- LocalLimit 20

+- 'Sort [spend_rank#678 ASC NULLS FIRST], true

+- Project [customer_id#28, total_spend#677, spend_rank#678]

+- Window [rank(_w0#683) windowSpecDefinition(_w0#683 DESC NULLS LAST, specifiedWindowFrame(RowFrame, unboundedPrecedingS(), currentRowS())) AS spend_rank#678], [_w0#683 DESC NULLS LAST]

+- Aggregate [customer_id#28], [customer_id#28, sum(total_price_usd#50) AS total_spend#677, sum(total_price_usd#50) AS _w0#683]

+- Project [customer_id#28, total_price_usd#50]

+- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv

```

== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- TakeOrderedAndProject(limit=20, orderBy=[spend_rank#678 ASC NULLS FIRST], output=[customer_id#28,total_spend#677,spend_rank#678])
   +- Project [customer_id#28, total_spend#677, spend_rank#678]
      +- Window [rank(_w0#683) windowSpecdefinition(_w0#683 DESC NULLS LAST, specifiedwindowframe(RowFrame, unboundedpreceding$(), c
urrentrow$())) AS spend_rank#678], [_w0#683 DESC NULLS LAST]
         +- Sort [_w0#683 DESC NULLS LAST], false, 0
            +- Exchange SinglePartition, ENSURE_REQUIREMENTS, [plan_id=785]
               +- HashAggregate(keys=[customer_id#28], functions=[sum(total_price_usd#50)], output=[customer_id#28, total_spend#677,
_w0#683])
                  +- Exchange hashpartitioning(customer_id#28, 8), ENSURE_REQUIREMENTS, [plan_id=782]
                     +- HashAggregate(keys=[customer_id#28], functions=[partial_sum(total_price_usd#50)], output=[customer_id#28, su
Terminal
+- FileScan csv [customer_id#28,total_price_usd#50] Batched: false, DataFilters: [], Format: CSV, Location:
InMemoryFileIndex(1 paths)[file:/home/yousef/spark_project/data/ecommerce.csv], PartitionFilters: [], PushedFilters: [], ReadSchema:
struct<customer_id:string,total_price_usd:double>

```

QUERY 10

```

104      "Q10 - High-value customers (subquery)": ""
105      SELECT *
106      FROM (
107          SELECT customer_id, SUM(total_price_usd) AS total_spend
108          FROM ecommerce
109          GROUP BY customer_id
110      ) customer_totals
111      WHERE total_spend > (
112          SELECT AVG(total_spend) FROM (
113              SELECT customer_id, SUM(total_price_usd) AS total_spend
114              FROM ecommerce
115              GROUP BY customer_id
116          )
117      )
118      ORDER BY total_spend DESC
119      LIMIT 10
120      ""
121 }

```

```
=====
Running Q10 via Spark SQL...
```

```
+-----+-----+
|customer_id|    total_spend|
+-----+-----+
| CUS-CSPL3|         4267.91|
| CUS-WXS3A|4043.9300000000003|
| CUS-OK609|         3888.2|
| CUS-A67K3|         3735.48|
| CUS-P1D03|3542.4300000000003|
| CUS-5XVL1|         3506.8|
| CUS-96P9R|         3495.37|
| CUS-Q9X3X|3467.6400000000003|
| CUS-A20PJ|         3422.48|
| CUS-V010N|3334.0200000000004|
+-----+-----+
```

```
Q10 Time: 11.411s
```

```
=== Q10 EXPLAIN ===
```

```
== Parsed Logical Plan ==
```

```
'GlobalLimit 10
```

```
+ 'LocalLimit 10
```

```
  +- 'Sort ['total_spend DESC NULLS LAST], true
```

```
    +- 'Project [*]
```

```
      +- 'Filter ('total_spend > scalar-subquery#716 [])
```

```
        : +- 'Project [unresolvedalias('AVG('total_spend), None)]
```

```
          SubqueryAlias __auto_generated_subquery_name
```

```
            +- 'Aggregate ['customer_id], ['customer_id, 'SUM('total_price_usd) AS total_spend#715]
```

```
              : +- 'UnresolvedRelation [ecommerce], [], false
```

```
            +- 'SubqueryAlias customer_totals
```

```
              +- 'Aggregate ['customer_id], ['customer_id, 'SUM('total_price_usd) AS total_spend#714]
```

```
                +- 'UnresolvedRelation [ecommerce], [], false
```

Activate Windows

0,... 38 more files) CSV

== Physical Plan ==

AdaptiveSparkPlan isFinalPlan=false

```
+ TakeOrderedAndProject(limit=10, orderBy=[total_spend#714 DESC NULLS LAST], output=[customer_id#28,total_spend#714])
+ Filter (isnotnull(total_spend#714) AND (total_spend#714 > Subquery subquery#716, [id=#968]))
  : +- Subquery subquery#716, [id=#968]
  :   +- AdaptiveSparkPlan isFinalPlan=false
  :     +- HashAggregate(keys=[], functions=[avg(total_spend#715)], output=[avg(total_spend)#720])
  :       +- Exchange SinglePartition, ENSURE_REQUIREMENTS, [plan_id=966]
  :         +- HashAggregate(keys=[], functions=[partial_avg(total_spend#715)], output=[sum#795, count#796L])
  :           +- HashAggregate(keys=[customer_id#732], functions=[sum(total_price_usd#754)], output=[total_spend#715])
  :             +- Exchange hashpartitioning(customer_id#732, 8), ENSURE_REQUIREMENTS, [plan_id=962]
  :               +- HashAggregate(keys=[customer_id#732], functions=[partial_sum(total_price_usd#754)], output=[customer_id#732, sum#798])
  :                 +- FileScan csv [customer_id#732,total_price_usd#754] Batched: false, DataFilters: [], Format: CSV,
  :                   Location: InMemoryFileIndex(1 paths)[file:/home/yousef/spark_project/data/ecommerce.csv], PartitionFilters: [], PushedFilters: [],
  :                   ReadSchema: struct<customer_id:string,total_price_usd:double>
  :                 +- HashAggregate(keys=[customer_id#28], functions=[sum(total_price_usd#50)], output=[customer_id#28, total_spend#714])
  :                   +- Exchange hashpartitioning(customer_id#28, 8), ENSURE_REQUIREMENTS, [plan_id=972]
  :                     +- HashAggregate(keys=[customer_id#28], functions=[partial_sum(total_price_usd#50)], output=[customer_id#28, sum#792])
  :                       +- FileScan csv [customer_id#28,total_price_usd#50] Batched: false, DataFilters: [], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/yousef/spark_project/data/ecommerce.csv], PartitionFilters: [], PushedFilters: [], ReadSchema: struct<customer_id:string,total_price_usd:double>
```

All SQL timings saved.

Join_optimization

```
=====
JOIN OPTIMIZATION: Broadcast Join vs Sort-Merge Join
=====

Broadcast Join count: 1000123
=== BROADCAST JOIN EXPLAIN ===
== Parsed Logical Plan ==
'Project [category#40, order_id#17, order_date#18, order_year#19, order_month#20, order_day#21, order_hour#22, order_minute#23, order_second#24, is_weekend#25, order_status#26, return_reason#27, customer_id#28, customer_name#29, gender#30, age#31, customer_segment#32, country#33, city#34, customer_loyalty_score#35, total_orders_by_customer#36, account_creation_date#37, product_id#38, product_name#39, ... 40 more fields]
+- Project [category#40, order_id#17, order_date#18, order_year#19, order_month#20, order_day#21, order_hour#22, order_minute#23, order_second#24, is_weekend#25, order_status#26, return_reason#27, customer_id#28, customer_name#29, gender#30, age#31, customer_segment#32, country#33, city#34, customer_loyalty_score#35, total_orders_by_customer#36, account_creation_date#37, product_id#38, product_name#39, ... 39 more fields]
  +- Join LeftOuter, (category#40 = category#520)
    :- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
Terminal ResolvedHint (strategy=broadcast)
  +- LogicalRDD [category#520, discount_rate#521], false
```

```
== Analyzed Logical Plan ==
category: string, order_id: string, order_date: timestamp, order_year: int, order_month: int, order_day: int, order_hour: int, order_minute: int, order_second: int, is_weekend: string, order_status: string, return_reason: string, customer_id: string, customer_name: string, gender: string, age: int, customer_segment: string, country: string, city: string, customer_loyalty_score: double, total_orders_by_customer: int, account_creation_date: date, product_id: string, product_name: string, ... 40 more fields
Project [category#40, order_id#17, order_date#18, order_year#19, order_month#20, order_day#21, order_hour#22, order_minute#23, order_second#24, is_weekend#25, order_status#26, return_reason#27, customer_id#28, customer_name#29, gender#30, age#31, customer_segment#32, country#33, city#34, customer_loyalty_score#35, total_orders_by_customer#36, account_creation_date#37, product_id#38, product_name#39, ... 40 more fields]
+- Project [category#40, order_id#17, order_date#18, order_year#19, order_month#20, order_day#21, order_hour#22, order_minute#23, order_second#24, is_weekend#25, order_status#26, return_reason#27, customer_id#28, customer_name#29, gender#30, age#31, customer_segment#32, country#33, city#34, customer_loyalty_score#35, total_orders_by_customer#36, account_creation_date#37, product_id#38, product_name#39, ... 39 more fields]
  +- Join LeftOuter, (category#40 = category#520)
    :- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
    +- ResolvedHint (strategy=broadcast)
      +- LogicalRDD [category#520, discount_rate#521], false
```



```

== Optimized Logical Plan ==
Project [category#40, order_id#17, order_date#18, order_year#19, order_month#20, order_day#21, order_hour#22, order_minute#23, order_second#24, is_weekend#25, order_status#26, return_reason#27, customer_id#28, customer_name#29, gender#30, age#31, customer_segment#32, country#33, city#34, customer_loyalty_score#35, total_orders_by_customer#36, account_creation_date#37, product_id#38, product_name#39, ... 40 more fields]
+- Join LeftOuter, (category#40 = category#520), rightHint=(strategy=broadcast)
   :- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
   +- Filter isNotNull(category#520)
      +- LogicalRDD [category#520, discount_rate#521], false

== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- Project [category#40, order_id#17, order_date#18, order_year#19, order_month#20, order_day#21, order_hour#22, order_minute#23, order_second#24, is_weekend#25, order_status#26, return_reason#27, customer_id#28, customer_name#29, gender#30, age#31, customer_segment#32, country#33, city#34, customer_loyalty_score#35, total_orders_by_customer#36, account_creation_date#37, product_id#38, product_name#39, ... 40 more fields]
   s-yousef.shawky@zewailcity.edu.eg gory#40], [category#520], LeftOuter, BuildRight, false
   id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] Batched: false, DataFilters: [], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/yousef/spark_project/data/ecommerce.csv], PartitionFilters: [], PushedFilters: [], ReadSchema: struct<order_id:string,order_date:timestamp,order_year:int,order_month:int,order_day:int,order_ho...
   +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, false]),false), [plan_id=190]
      +- Filter isNotNull(category#520)
         +- Scan ExistingRDD[category#520,discount_rate#521]

```

Activate Windows

```

Sort-Merge Join count: 1000123
=== SORT-MERGE JOIN EXPLAIN ===
== Parsed Logical Plan ==
'Project [category#40, order_id#17, order_date#18, order_year#19, order_month#20, order_day#21, order_hour#22, order_minute#23, order_second#24, is_weekend#25, order_status#26, return_reason#27, customer_id#28, customer_name#29, gender#30, age#31, customer_segment#32, country#33, city#34, customer_loyalty_score#35, total_orders_by_customer#36, account_creation_date#37, product_id#38, product_name#39, ... 40 more fields]
+- Project [category#40, order_id#17, order_date#18, order_year#19, order_month#20, order_day#21, order_hour#22, order_minute#23, order_second#24, is_weekend#25, order_status#26, return_reason#27, customer_id#28, customer_name#29, gender#30, age#31, customer_segment#32, country#33, city#34, customer_loyalty_score#35, total_orders_by_customer#36, account_creation_date#37, product_id#38, product_name#39, ... 39 more fields]
   +- Join LeftOuter, (category#40 = category#520)
      :- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
      +- LogicalRDD [category#520, discount_rate#521], false

```



```

== Analyzed Logical Plan ==
category: string, order_id: string, order_date: timestamp, order_year: int, order_month: int, order_day: int, order_hour: int, order_minute: int, order_second: int, is_weekend: string, order_status: string, return_reason: string, customer_id: string, customer_name: string, gender: string, age: int, customer_segment: string, country: string, city: string, customer_loyalty_score: double, total_orders_by_customer: int, account_creation_date: date, product_id: string, product_name: string, ... 40 more fields
Project [category#40, order_id#17, order_date#18, order_year#19, order_month#20, order_day#21, order_hour#22, order_minute#23, order_second#24, is_weekend#25, order_status#26, return_reason#27, customer_id#28, customer_name#29, gender#30, age#31, customer_segment#32, country#33, city#34, customer_loyalty_score#35, total_orders_by_customer#36, account_creation_date#37, product_id#38, product_name#39, ... 40 more fields]
+- Project [category#40, order_id#17, order_date#18, order_year#19, order_month#20, order_day#21, order_hour#22, order_minute#23, order_second#24, is_weekend#25, order_status#26, return_reason#27, customer_id#28, customer_name#29, gender#30, age#31, customer_segment#32, country#33, city#34, customer_loyalty_score#35, total_orders_by_customer#36, account_creation_date#37, product_id#38, product_name#39, ... 39 more fields]
    +- Join LeftOuter, (category#40 = category#520)
        :- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
        s-yousef.shawky@zewailcity.edu.eg #520, discount_rate#521], false

```

```

== Optimized Logical Plan ==
Project [category#40, order_id#17, order_date#18, order_year#19, order_month#20, order_day#21, order_hour#22, order_minute#23, order_second#24, is_weekend#25, order_status#26, return_reason#27, customer_id#28, customer_name#29, gender#30, age#31, customer_segment#32, country#33, city#34, customer_loyalty_score#35, total_orders_by_customer#36, account_creation_date#37, product_id#38, product_name#39, ... 40 more fields]
+- Join LeftOuter, (category#40 = category#520)
    :- Relation [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] csv
    +- Filter isNotNull(category#520)
        +- LogicalRDD [category#520, discount_rate#521], false

== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- Project [category#40, order_id#17, order_date#18, order_year#19, order_month#20, order_day#21, order_hour#22, order_minute#23, order_second#24, is_weekend#25, order_status#26, return_reason#27, customer_id#28, customer_name#29, gender#30, age#31, customer_segment#32, country#33, city#34, customer_loyalty_score#35, total_orders_by_customer#36, account_creation_date#37, product_id#38, product_name#39, ... 40 more fields]
    +- SortMergeJoin [category#40], [category#520], LeftOuter
        :- Sort [category#40 ASC NULLS FIRST], false, 0
        : +- Exchange hashpartitioning(category#40, 8), ENSURE_REQUIREMENTS, [plan_id=392]
        :   +- FileScan csv [order_id#17,order_date#18,order_year#19,order_month#20,order_day#21,order_hour#22,order_minute#23,order_second#24,is_weekend#25,order_status#26,return_reason#27,customer_id#28,customer_name#29,gender#30,age#31,customer_segment#32,country#33,city#34,customer_loyalty_score#35,total_orders_by_customer#36,account_creation_date#37,product_id#38,product_name#39,category#40,... 38 more fields] Batched: false, DataFilters: [], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/yousef/spark_pr
        Ubuntu22.04.5 LTS amd64 sv], PartitionFilters: [], PushedFilters: [], ReadSchema: struct<order_id:string,order_date:timestamp,order_year:int,order_month:int,order_day:int,order_ho...
        +- Sort [category#520 ASC NULLS FIRST], false, 0
        +- Exchange hashpartitioning(category#520, 8), ENSURE_REQUIREMENTS, [plan_id=393]
        +- Filter isNotNull(category#520)
        +- Scan ExistingRDD[category#520,discount_rate#521]

```

Activate Windows


```
--- JOIN PERFORMANCE COMPARISON ---
broadcast_join_time: 11.128s
sort_merge_join_time: 11.454s
```

```
=====
CACHING: With Cache vs Without Cache
=====
```

```
+-----+-----+
| category|sum(total_price_usd)|
+-----+-----+
|      None| 8.782867775999954E7|
|    Health|5.7467925550000004E7|
|Electronics|1.3986598694000006E8|
|    Sports| 7.291747085000001E7|
|   Clothing| 4.520579590000001E7|
+-----+-----+
```

```
+-----+-----+
| category|count(1)|
+-----+-----+
```

s-yousef.shawky@zewailcity.edu.eg

```
|Electronics| 200019|
|    Sports| 200397|
|   Clothing| 199544|
+-----+-----+
```

```
+-----+-----+
| category|sum(total_price_usd)|
```

LibreOffice Writer

```
|      None| 8.782867775999954E7|
|    Health|5.7467925550000004E7|
|Electronics|1.3986598694000006E8|
|    Sports| 7.291747085000001E7|
|   Clothing| 4.520579590000001E7|
+-----+-----+
```

```
+-----+-----+
| category|count(1)|
+-----+-----+
```

```
|      None| 200589|
|    Health| 199574|
|Electronics| 200019|
|    Sports| 200397|
|   Clothing| 199544|
+-----+-----+
```

===== PARTITIONING: Impact on Query Performance =====

country	sum(total_price_usd)
Australia	4.0331338920000106E7
United States	4.023669859999991E7
United Kingdom	4.034049741999993E7
Netherlands	4.054504649000018E7
Italy	4.047603399999995E7
Belgium	4.041667991999994E7
Spain	4.037728950999999E7
France	4.023041559999984E7
Germany	4.012906336000008E7
Canada	4.020279318000002E7

country	sum(total_price_usd)
Germany	4.0129063359999E7
Belgium	4.041667992000048E7
Spain	4.037728950999957E7
France	4.023041560000009E7
Canada	4.020279317999988E7
United Kingdom	4.034049742000009E7
Australia	4.033133892000024E7
Netherlands	4.054504648999942E7
Italy	4.047603399999975E7
United States	4.0236698600000024E7

===== FILE FORMAT: CSV vs Parquet =====

category	sum(total_price_usd)
Hone	8.78286777599993E7
Health	5.746792554999926E7
Electronics	1.3986598693999696E8
Sports	7.291747084999952E7
Clothing	4.520579589999961E7

category	sum(total_price_usd)
Hone	8.782867775999954E7
Health	5.7467925550000004E7
Electronics	1.3986598694000006E8
Sports	7.291747085000001E7
Clothing	4.520579590000001E7

Cluster Configuration and Scalability Tests

1.

```
yousef@yousef-VirtualBox:~/spark_project$ python3 scalability_test.py 2>&1 | tee results/scalability_output.log
26/04/25 14:34:41 WARN Utils: Your hostname, yousef-VirtualBox resolves to a loopback address: 127.0.1.1; using 10.0.2.15 instead (on interface enp0s3)
26/04/25 14:34:41 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/04/25 14:34:54 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable

=====
Testing config: {'partitions': 4, 'memory': '1g'}
=====

=== CLUSTER CONFIGURATION ===
Shuffle Partitions: 4
Driver Memory: 1g

=== SCHEMA ===
root
  order_id: string (nullable = true)
```

```
159
160 +-----+-----+
161 | category|sum(total_price_usd)|
162 +-----+-----+
163 |      Home| 8.782867775999954E7|
164 |    Health|5.7467925550000004E7|
165 |Electronics|1.3986598694000006E8|
166 |    Sports| 7.291747085000001E7|
167 | Clothing| 4.5205795900000001E7|
168 +-----+-----+
169
170 Execution Time: 32.152s
171
```


2.

```
171
172 =====
173 Testing config: {'partitions': 8, 'memory': '2g'}
174 =====
175
176 === CLUSTER CONFIGURATION ===
177 Shuffle Partitions: 8
178 Driver Memory: 2g
179
180 === SCHEMA ===
181 root
182 |-- order_id: string (nullable = true)
183 |-- order_date: timestamp (nullable = true)
184 |-- order_year: integer (nullable = true)
185 |-- order_month: integer (nullable = true)
```

```
294
295
296 +-----+-----+
297 |  category|sum(total_price_usd)|
298 +-----+-----+
299 |      Home| 8.782867775999954E7|
300 |    Health|5.7467925550000004E7|
301 |Electronics|1.3986598694000006E8|
302 |    Sports| 7.291747085000001E7|
303 |  Clothing| 4.520579590000001E7|
304 +-----+-----+
305
306 Execution Time: 14.328s
307
```

3.

```
307
308 =====
309 Testing config: {'partitions': 16, 'memory': '4g'}
310 =====
311
312 === CLUSTER CONFIGURATION ===
313 Shuffle Partitions: 16
314 Driver Memory: 4g
315
316 === SCHEMA ===
317 root
318 |-- order_id: string (nullable = true)
319 |-- order_date: timestamp (nullable = true)
320 |-- order_year: integer (nullable = true)
321 |-- order_month: integer (nullable = true)
322 |-- order_day: integer (nullable = true)
323 |-- order_hour: integer (nullable = true)
324 |-- order_minute: integer (nullable = true)
```

```
405
406 +-----+-----+
407 | category|sum(total_price_usd)|
408 +-----+-----+
409 |      Home| 8.782867775999954E7|
410 |    Health|5.7467925550000004E7|
411 |Electronics|1.3986598694000006E8|
412 |    Sports| 7.291747085000001E7|
413 |  Clothing| 4.520579590000001E7|
414 +-----+-----+
415
416 Execution Time: 13.865s
417
```

```
417
418 === SCALABILITY RESULTS ===
419 {'partitions': 4, 'memory': '1g', 'time': 32.152283906936646}
420 {'partitions': 8, 'memory': '2g', 'time': 14.327798128128052}
421 {'partitions': 16, 'memory': '4g', 'time': 13.865058898925781}
```


Optimization Results

```
Optimization timings saved.
yousef@yousef-VirtualBox:~/spark_project$ python3 results/compare_results.py

=====
PERFORMANCE COMPARISON TABLE
=====
Query   RDD (s)   DataFrame (s)   SQL (s)   Best
-----
Q1      104.042   10.717          8.803     SQL
Q2      112.619   12.166          9.503     SQL
Q3      108.797   10.006          8.350     SQL
Q4      105.537   9.966           8.159     SQL
Q5      127.060   13.238          11.008    SQL
Q6      109.251   9.871           9.298     SQL
Q7      144.148   9.029           8.018     SQL
Q8      136.456   13.921          11.101    SQL
Q9      233.551   16.904          13.157    SQL
Q10     168.341   33.789          11.411    SQL
=====

Optimization Results:
broadcast_join_time: 11.128s
sort_merge_join_time: 11.454s
without_cache_time: 16.299s
with_cache_time: 3.702s
default_partitions_time: 7.410s
repartitioned_time: 9.633s
parquet_read_time: 3.504s
csv read time: 24.110s
```

Github Repository:

<https://github.com/mohamed1232005/Spark-Rdd-Vs-Dataframe-Sql-Performance-Analysis-Ecommerce>